

TEMA 2

MÉTODOS SUPERVISADOS

María N. Moreno García
Departamento de Informática y Automática
Grupo de Investigación en Minería de Datos
mida.usal.es

Asignatura: **Minería de Datos**

Índice

1. Introducción
2. Clasificación
3. Métodos de selección de características
4. Evaluación de los clasificadores
5. Predicción de valores

1. INTRODUCCIÓN

Introducción

Los métodos supervisados se aplican para predecir el valor de un atributo (etiqueta) de un conjunto de datos, conocidos otros atributos (atributos descriptivos)

Especializaciones del modelado predictivo

Clasificación

- El atributo etiqueta toma valores discretos (clases)
- Se usa para establecer una clase para cada registro de un conjunto de datos

Predicción de valores

- La etiqueta toma valores continuos
- El modelo se utiliza para estimar un valor numérico continuo que se asocia con un registro de una base de datos

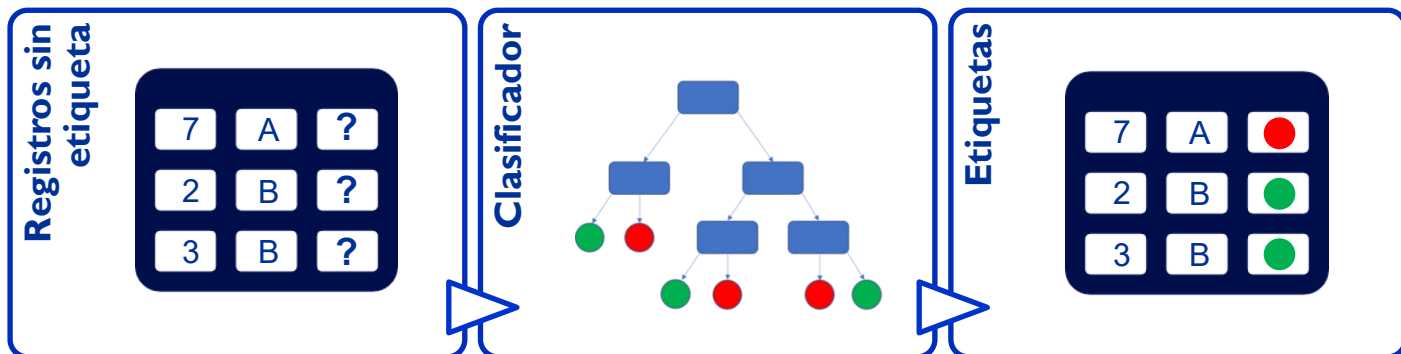
2. CLASIFICACIÓN

Clasificación

- Inducción y uso de un clasificador



Construcción de un clasificador a partir de registros clasificados



Uso del clasificador con registros sin etiqueta

Clasificación

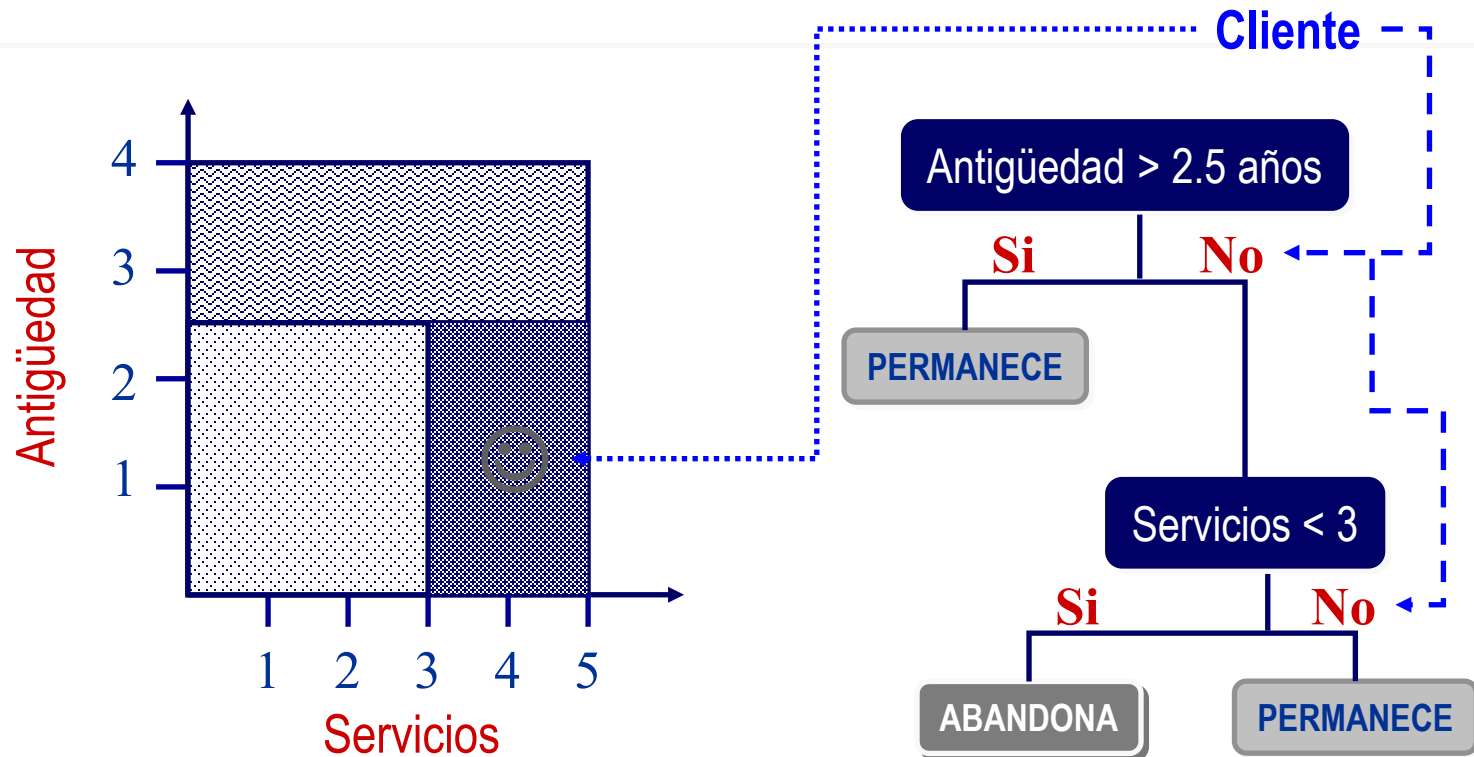
Inducción de árboles

Con ésta técnica se construye un modelo predictivo en forma de árbol de decisión

- Proceso:
 - El algoritmo comienza identificando la variable más importante (aquella que se considera que influye más en la clasificación) y se le asocia el nodo raíz del árbol
 - Todos los registros del conjunto de entrenamiento se chequean frente a esa condición. Los que satisfacen la condición se colocan en la rama izquierda del árbol y los que no en la derecha (para condiciones binarias)
 - A continuación, se decide cual es la siguiente variable en importancia y se repite el proceso
- Los puntos de decisión son los **nodos del árbol** y los puntos finales donde se recogen las observaciones son las **hojas**

Clasificación

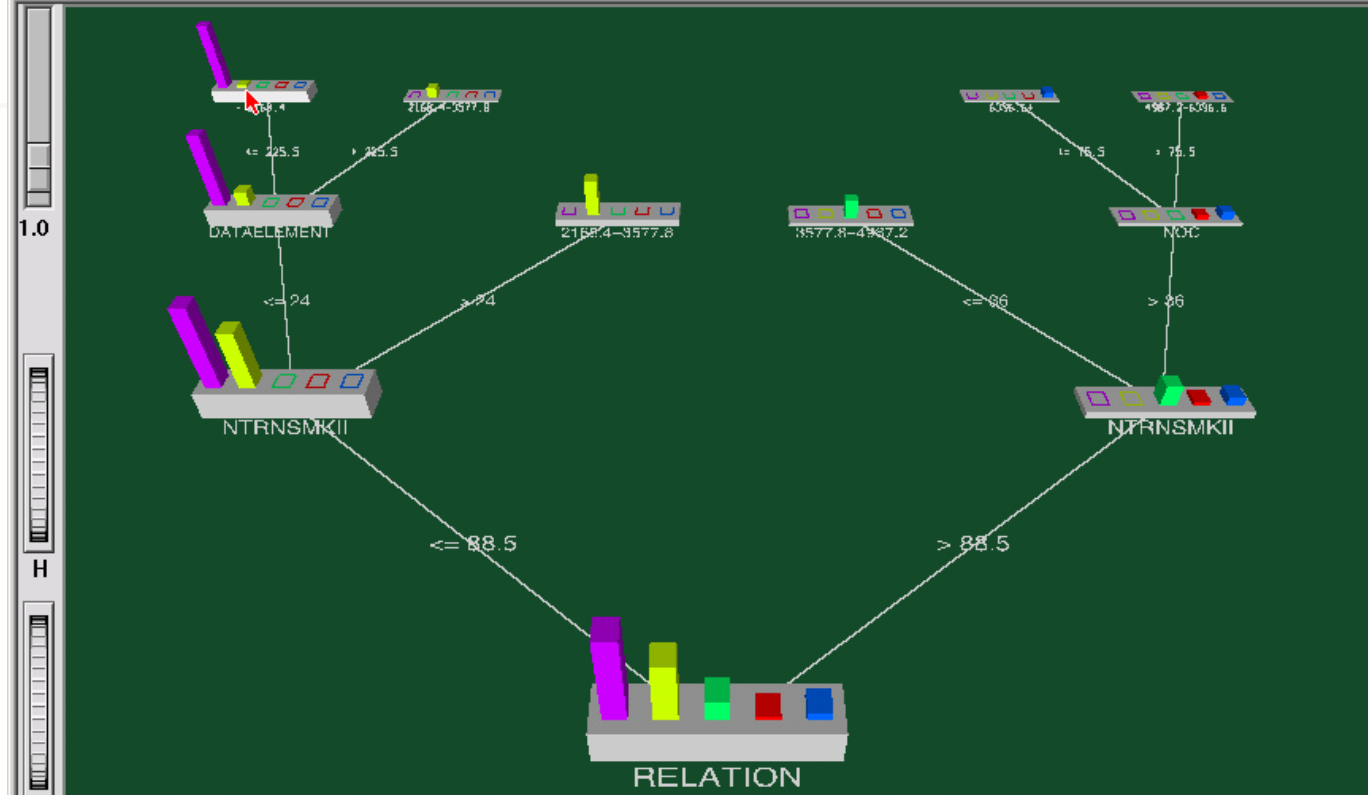
Inducción de árboles



Clasificación

Inducción de árboles

:RELATION<= 88.5:NTRNSMKII<= 24:DATAELEMENT<= 225.5:
Subtree weight:21.00, purity:88.10



Visualización tridimensional de un modelo de inducción de árboles

Clasificación

Inducción de árboles

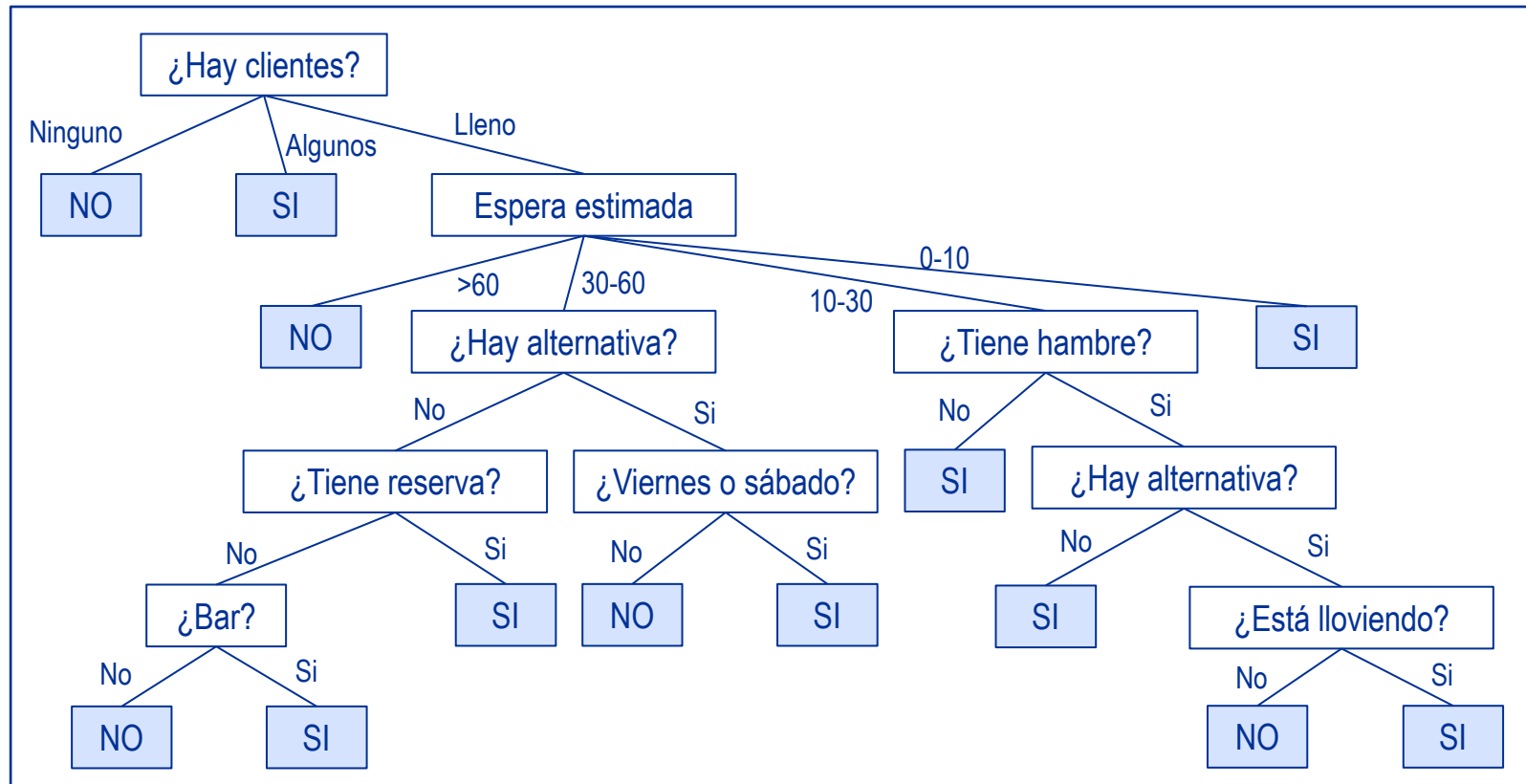
- **Ejemplo:** resolver el problema de decidir si se espera o no a que le asignen una mesa en un restaurante
 - **Etiqueta:** esperarán, **valores de la etiqueta:** SI, NO
 - **Atributos:** alternativa, bar, viernes/sábado, tener hambre, clientes, precio, lluvia, reserva, tipo, tiempo de espera

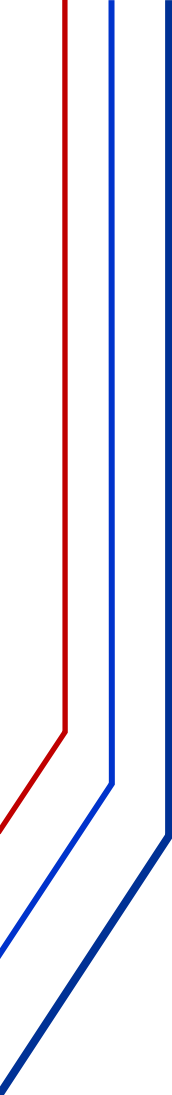
Ejemplo	Atributos										Etiqueta Esperarán
	Alt	Bar	Vier	Ham	Pat	Precio	Lluvia	Res	Tipo	Est	
X_1	Si	No	No	Si	Alg.	\$\$\$	No	Si	Fran.	0-10	Si
X_2	Si	No	No	Si	Lleno	\$	No	No	Tai.	30-60	No
X_3	No	Si	No	No	Alg.	\$	No	No	Ham.	0-10	Si
X_4	Si	No	Si	Si	Lleno	\$	No	No	Tai.	10-30	Si
X_5	Si	No	Si	No	Lleno	\$\$\$	No	Si	Fran.	>60	No
X_6	No	Si	No	Si	Alg.	\$\$	Si	Si	Ital.	0-10	Si
X_7	No	Si	No	No	Nin.	\$	Si	No	Ham.	0-10	No
X_8	No	No	No	Si	Alg.	\$\$	Si	Si	Tai.	0-10	Si
X_9	No	Si	Si	No	Lleno	\$	Si	No	Ham.	>60	No
X_{10}	Si	Si	Si	Si	Lleno	\$\$\$	No	Si	Ital.	10-30	No
X_{11}	No	No	No	No	Nin.	\$	No	No	Tai.	0-10	No
X_{12}	Si	Si	Si	Si	Lleno	\$	No	No	Ham.	30-60	Si

Clasificación

Inducción de árboles

- Una posible solución al problema podría ser la ofrecida por el árbol de decisión de la figura siguiente





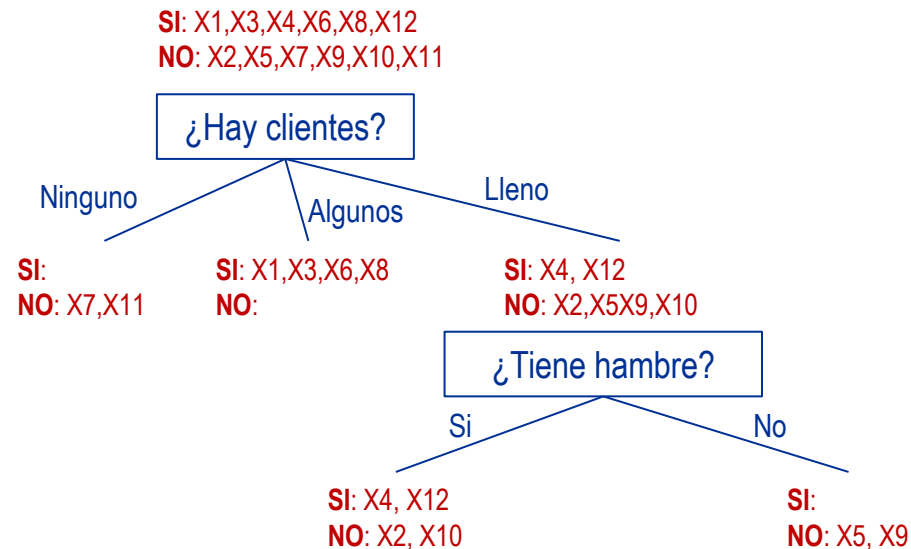
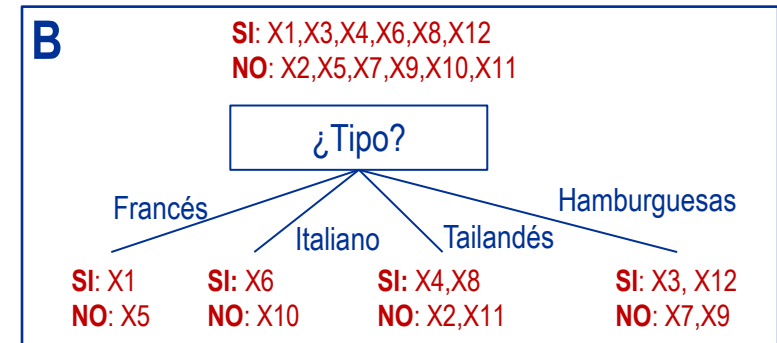
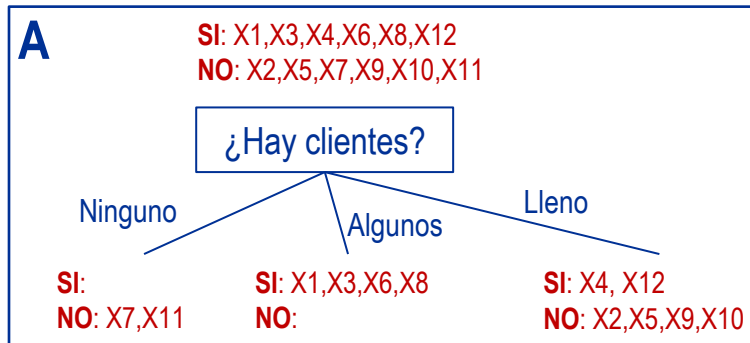
Clasificación

Inducción de árboles

- La mejor solución es la del árbol más pequeño
- Procedimiento de obtención del árbol
 - Se selecciona el atributo que implique la mayor diferenciación en la clasificación
 - Si en alguna de las ramas resultantes de la evaluación de ese atributo no se da la clasificación completa se repite el proceso con los atributos restantes

Clasificación

Inducción de árboles



Clasificación

Inducción de árboles

- Selección de los atributos que influyen en la clasificación
 - **Entropía:** Medida basada en la cantidad esperada de información que proporciona el atributo

$$H(P(c_1), \dots, P(c_n)) = \sum_{i=1}^n -P(c_i) \log_2 P(c_i)$$

$P(c_i)$: probabilidad de una clase

n : número de clases posibles (nº de valores posibles de la etiqueta)

La entropía es 1 cuando las probabilidades de las clases son iguales

- **Pureza** de un nodo del árbol

$$\text{Pureza} = 1 - \text{entropía}$$

La pureza máxima se obtiene cuando la probabilidad de una clase es del 100% y las probabilidades del resto son del 0% (clasificación completa)

Clasificación

Inducción de árboles

- **Medidas de impureza**

Entropía

$$H = \sum_j -p_j \log_2 p_j$$

Índice
Gini

$$G = 1 - \sum_j (p_j)^2$$

DKM

$$DKM = 2 \left(\prod_j p_j \right)^{1/2}$$

p_j : probabilidad de la clase j

Clasificación

Inducción de árboles

- **Criterios de partición**

Utilizados para seleccionar el mejor atributo para dividir el conjunto de registros

En el caso de árboles binarios, si D es el conjunto de entrenamiento que se divide en dos subconjuntos D_Y y D_N , n el número de elementos de D y n_Y y n_N el número de elementos de las particiones D_Y y D_N respectivamente

- **Ganancia de información**

- Diferencia en la entropía antes y después de realizar la partición

$$Gain = H(D) - H(D_Y, D_N)$$

Donde $H(D_Y, D_N)$ es la media ponderada de la entropía de cada partición del conjunto D en dos subconjuntos D_Y y D_N

Clasificación

Inducción de árboles

- **Criterios de partición**

- **Índice de ganancia**

- Divide la ganancia de información entre un valor llamado “*Split Information*” (SI) relativo a los valores del atributo considerado para hacer la partición

$$SI(at) = - \sum_{i=1}^K \frac{|T_i|}{|T|} \log_2 \frac{|T_i|}{|T|}$$

Donde K es el número de valores del atributo, T_i es el número de ejemplos para el i ésimo valor del atributo y T el número total de ejemplos

- **Criterio CART**

$$CART(D_Y, D_N) = 2 \frac{n_Y}{n} \frac{n_N}{n} \sum_{i=1}^k |P(c_i|D_Y) - P(c_i|D_N)|$$

Donde $P(c_i|D_Y)$ y $P(c_i|D_N)$ es la probabilidad de la clase i en cada partición

Clasificación

Inducción de árboles

- **Poda del árbol de decisión**

Consiste en evitar la separación repetitiva de atributos que no tienen evidente relevancia, aún cuando los datos de ese nodo del árbol no estén clasificados convenientemente

- Un atributo irrelevante divide el conjunto de entrenamiento en subconjuntos que tienen aproximadamente las mismas proporciones de cada una de las clases
- Se realizan pruebas estadísticas para determinar cuando los subárboles no son significativamente mejores que el nodo del que parten
- El crecimiento del árbol se detiene cuando la ganancia en la información obtenida con la partición de un conjunto se encuentra por debajo de una cantidad umbral.
- La poda se utiliza para evitar el problema de **sobreadaptación** (*overfitting*) que consiste en encontrar una regularidad irrelevante en los datos

Clasificación

Inducción de árboles

ALGORITMO Partición(N :nodo, E :cjto. de ejemplos)

SI todos los ejemplos E son de la misma clase c **ENTONCES**

Asignar la clase c al nodo N .

SALIR; // Esta rama es pura, ya no hay que seguir partiendo. N es hoja.

SI NO:

Particiones := generar posibles particiones.

MejorPartición := seleccionar la mejor partición según el criterio de partición.

PARA CADA condición i de la partición elegida.

Añadir un nodo hijo i a N y asignar los ejemplos consistentes a cada hijo (E_i).

Partición(i , E_i). // Realizar el mismo procedimiento global con cada hijo.

FIN PARA

FIN SI

FIN ALGORITMO

Para clasificar un cjto de ejemplos E , se invoca con la llamada Partición(R , E), donde R es un nodo raíz de un árbol por empezar.

Algoritmo básico de aprendizaje de árboles de decisión

Clasificación

Inducción de árboles

- **Algoritmos**

- **ID3** (*Induction of Decision Trees*)[Quinlan, 1986]
 - Particiones basadas en **ganancia de información**
 - No puede utilizarse con atributos numéricos continuos
- **C4.5** [Quinlan, 1993]
 - Usa el **índice de ganancia** como criterio de partición
 - Permite post-poda del árbol
- **CART** (*Classification and Regression Trees*)[Breiman et al., 1984]
 - Utiliza su propia medida para realizar la partición
 - La poda se realiza mediante una simplificación iterativa del árbol minimizando la medida *complejidad del coste*
- **ASSISTANT** [Cestnik et al., 1987]
 - Árbol binario que usa el criterio de **ganancia de información** sobre un subconjunto de valores de los atributos
 - Incluye algoritmos para elegir el mejor conjunto de entrenamiento y reduce el árbol mediante la poda para maximizar la exactitud de la clasificación del conjunto de prueba

Clasificación

Tablas de decisión

Muestran correlaciones entre pares de atributos a diferentes niveles

- Una tabla de decisión se induce automáticamente mediante el cálculo de probabilidades a partir de los registros del conjunto de entrenamiento:
 - Los atributos continuos se separan en intervalos discretos
 - Se determina la probabilidad previa de la etiqueta: probabilidad de cada clase sin considerar el resto de atributos
 - Sobre los ejes de la tabla aparecen representados el número de intervalos discretos de valores para cada par de atributos

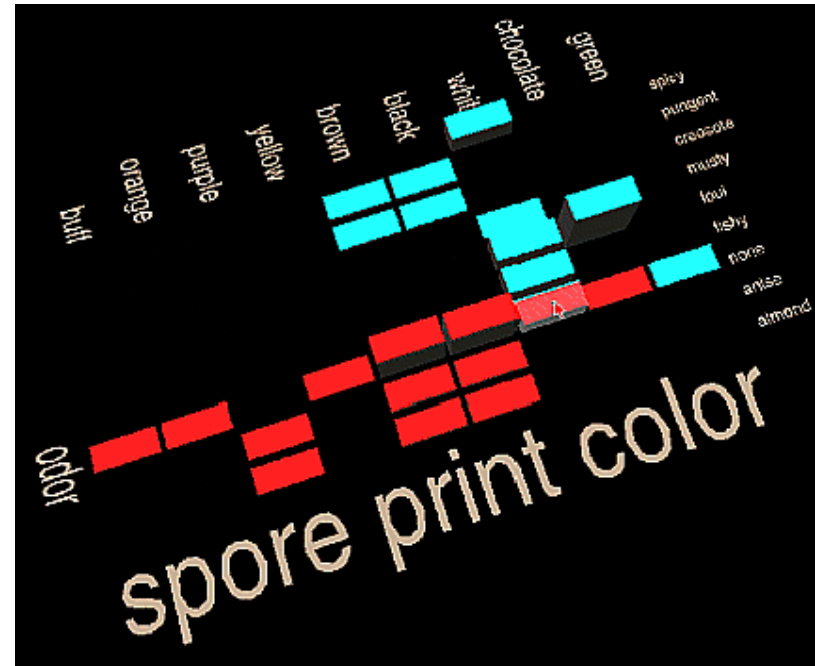


***Probabilidad
previa de la
etiqueta***

Clasificación

Tablas de decisión

- En el interior de la tabla se muestra la distribución de probabilidad para cada combinación de valores de los atributos (bloques de la tabla)
- En cada bloque se muestra, mediante colores diferentes, la proporción de registros que tiene de cada una de las clases
- Los bloques de un solo color únicamente tienen registros de una clase

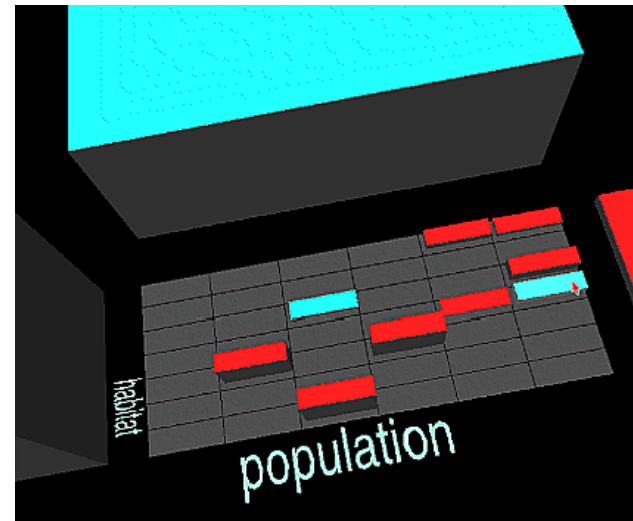
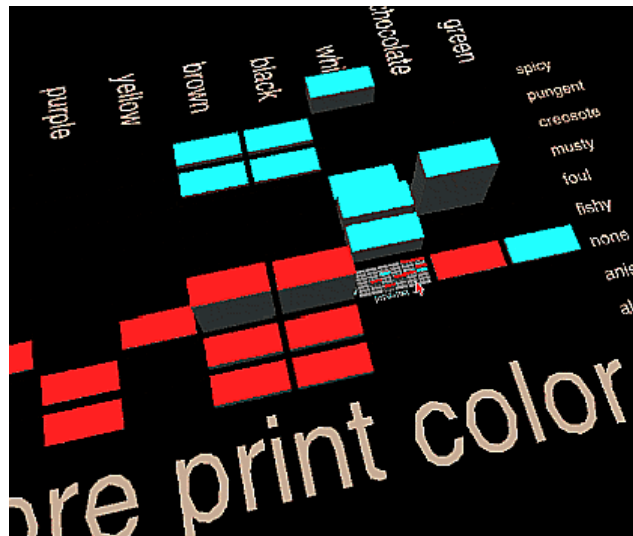


Visualización de una tabla de decisión

Clasificación

Tablas de decisión

- En el primer nivel se muestran los atributos que proporcionan la mejor clasificación
- En los bloques donde no se ha producido la clasificación completa (aparece más de un color) se puede acceder al siguiente nivel de la tabla en el que se relacionan otros dos atributos



Clasificación

Clasificadores bayesianos

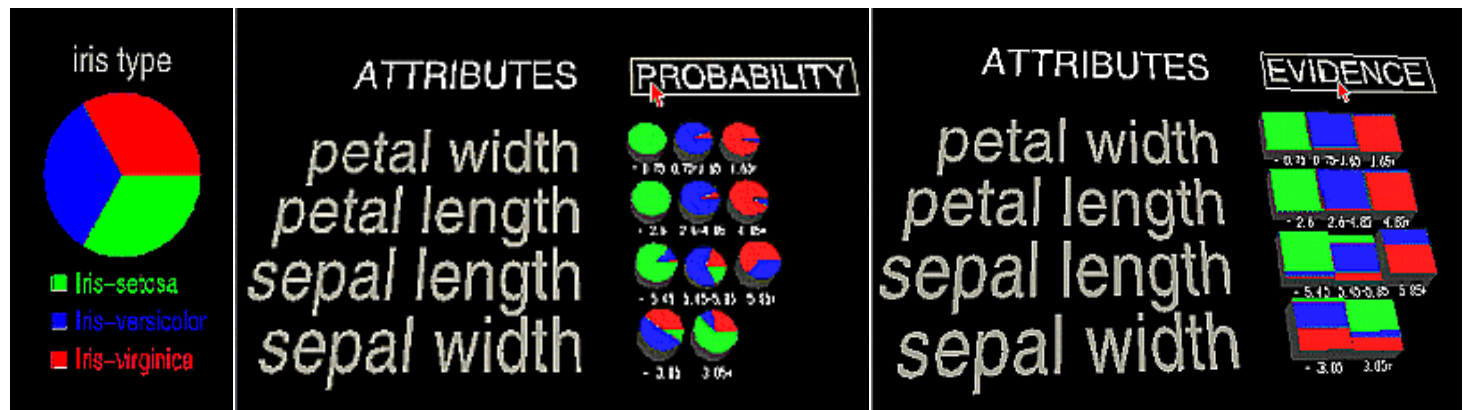
- Modelo de representación del conocimiento con incertidumbre
- Las relaciones de dependencia entre las variables del modelo se representan mediante un grafo acíclico
- Se basan en la teoría de la probabilidad y el teorema de Bayes
 - **Probabilidad previa o a priori**: Probabilidad de una hipótesis h sin considerar ninguna observación (probabilidad de los valores de la etiqueta sin considerar los valores de los atributos)
 - **Probabilidad a posteriori**: Probabilidad de una hipótesis h dada una observación O (probabilidad relativa de que cada valor de un atributo condicione un valor de la etiqueta)
 - **Teorema de Bayes**

$$P(h|O) = \frac{P(O|h) P(h)}{P(O)}$$

Clasificación

Clasificadores bayesianos

- En problemas de clasificación
 - **Probabilidad previa:** probabilidad de las clases o valores de la etiqueta sin considerar los valores de los atributos
 - **Probabilidad a posteriori:** probabilidad de una clase dado un valor (o intervalo de valores) de un atributo
 - **Probabilidad de la evidencia:** probabilidad de un valor (o intervalo de valores) del atributo dada una clase



Clasificación

Clasificadores bayesianos

- **Hipótesis MAP** (*Maximum A Posteriori*)

- Teorema de Bayes para un problema de clasificación con una variable clase **C** y un conjunto de atributos descriptivos $\{A_1 \dots A_n\}$

$$P(C | A_1 \dots A_n) = \frac{P(A_1 \dots A_n | C) P(C)}{P(A_1 \dots A_n)}$$

- Si **C** tiene **k** posibles valores $c_1 \dots c_k$, la hipótesis que se elige como resultado de la clasificación será aquella que tenga la máxima probabilidad a posteriori dados los atributos: **hipótesis máxima a posteriori** o **hipótesis MAP**

$$c_{MAP} = \arg \max P(A_1, \dots, A_n | c) P(c)$$

- El problema de este método es la alta complejidad computacional

Clasificación

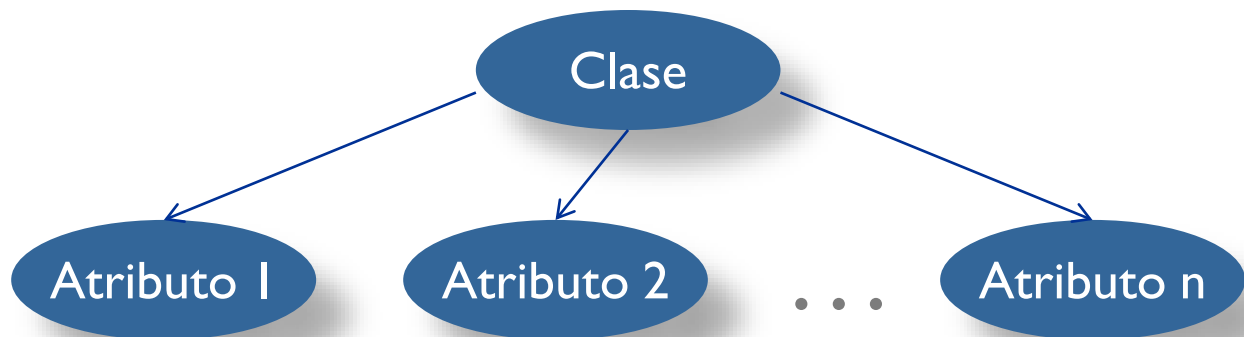
Clasificadores bayesianos

- **Naïve Bayes**

- Es el modelo más simple de clasificación con redes bayesianas basado en la suposición de que todos los atributos son independientes conocido el valor de la variable clase

$$c_{MAP} = \arg \max P(A_1, \dots, A_n | c) P(c) = \arg \max P(c) \prod_i P(A_i | c)$$

- La hipótesis de independencia da lugar a un modelo gráfico probabilístico con la clase como nodo raíz y los atributos como nodos hoja



Clasificación

Clasificadores bayesianos

- **Probabilidades de los atributos**

- **Atributos discretos**

- Estimación por **máxima verosimilitud**

$$P(x_i|c) = \frac{n(x_i, c)}{n(c)}$$

$n(x_i, c)$: número de ejemplos de la clase c que tienen el valor del atributo x_i

$n(c)$: número de ejemplos de la clase c

- Estimación basada en la **ley de sucesión de Laplace**

$$P(x_i|c) = \frac{n(x_i, c) + 1}{n(c) + |\Omega_{X_i}|}$$

Ω_{X_i} : número de valores posibles del atributo X_i

- **Atributos continuos**

- Se supone que tienen una distribución normal con media μ y desviación estándar σ

$$P(x_i|c) \sim N(\mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Clasificación

Clasificadores bayesianos

- Ejercicio
 - Hallar la hipótesis MAP para una instancia con valores de los atributos $A1=h$, $A2=b$ y $A3=2$ a partir del conjunto de entrenamiento de la tabla siguiente

A1	A2	A3	Clase
h	a	1	SI
m	a	2	NO
m	b	3	NO
h	b	4	SI
m	b	2	SI
h	a	3	NO

Clasificación

Clasificadores bayesianos

- **Redes bayesianas [Pearl, 1988]**

- Una red bayesiana es una tupla $B=(G,\Theta)$, donde G es el grafo y Θ es el conjunto de distribuciones de probabilidad $P(X_i|Pa(X_i))$ para cada variable X_i desde $i=1$ hasta n , donde $Pa(X_i)$ representa los padres de la variable en el grafo G
- El modelo de dependencias del grafo considera la relación de independencia condicional (X es condicionalmente independiente de Y dado Z):

$$I(X,Y|Z) \Leftrightarrow P(X|Y,Z) = P(X|Z)$$

- Las relaciones de independencia de la red se pueden expresar almacenando la **distribución de probabilidad conjunta** de forma local a cada una de las variables del modelo:

$$P(X_1 \dots X_n) = \prod_i P(X_i|Pa(X_i))$$

Clasificación

Clasificadores bayesianos

- **Algoritmos de búsqueda**

- El aprendizaje de redes bayesianas para un conjunto de datos D consiste en encontrar, entre todos los grafos posibles, el grafo G que mejor represente el conjunto de dependencias/independencias entre los datos
- Se requiere calcular las probabilidades a posteriori de todos los grafos. La complejidad de este problema es NP-duro, por lo que no es factible una solución exacta y hay que recurrir a métodos de **búsqueda heurística**
 - Se establece una métrica de calidad que represente la adecuación de una red bayesiana a un conjunto de datos
 - Se busca la solución que maximice esa métrica mediante un procedimiento de optimización
- Algoritmos de búsqueda: K2, B, TAN, BAN ...

Clasificación

Clasificadores bayesianos

- **Algoritmo K2**

1. Se parte de un conjunto de variables de entrada ordenadas
2. Inicialmente el conjunto de padres para una variable dada es el conjunto vacío
3. Para cada variable X_i
 - a. Se calcula la ganancia* que se produce en la métrica al introducir una variable X_j como padre
 - b. Se realiza el paso anterior para todo X_j tal que $j < i$
 - c. Se selecciona la variable con mayor ganancia
 - d. Se aplican los pasos a - c para seleccionar el resto de los padres

* La ganancia se calcula en primer lugar como la razón de la métrica (utilizada para evaluar la red) para la red con X_i sin padres y con el primer padre. En sucesivas evaluaciones es la razón de la métrica para la red con X_i con los padres de la iteración anterior y la métrica para la red con X_i con los padres de la iteración actual

Clasificación

Clasificadores bayesianos

- **Algoritmo TAN** (*Tree Augmented Naïve Bayes*)
 - Extensión del clasificador NB
 - Se construye una red bayesiana en la que se le da un tratamiento especial a la clase
 - En lugar de considerar las variables independientes dada la clase, se consideran ciertas dependencias entre ellas
 - Utilizan el concepto de **información mutua** para medir la cantidad de información que la variable Y proporciona sobre la variable X dado el valor de la variable clase C

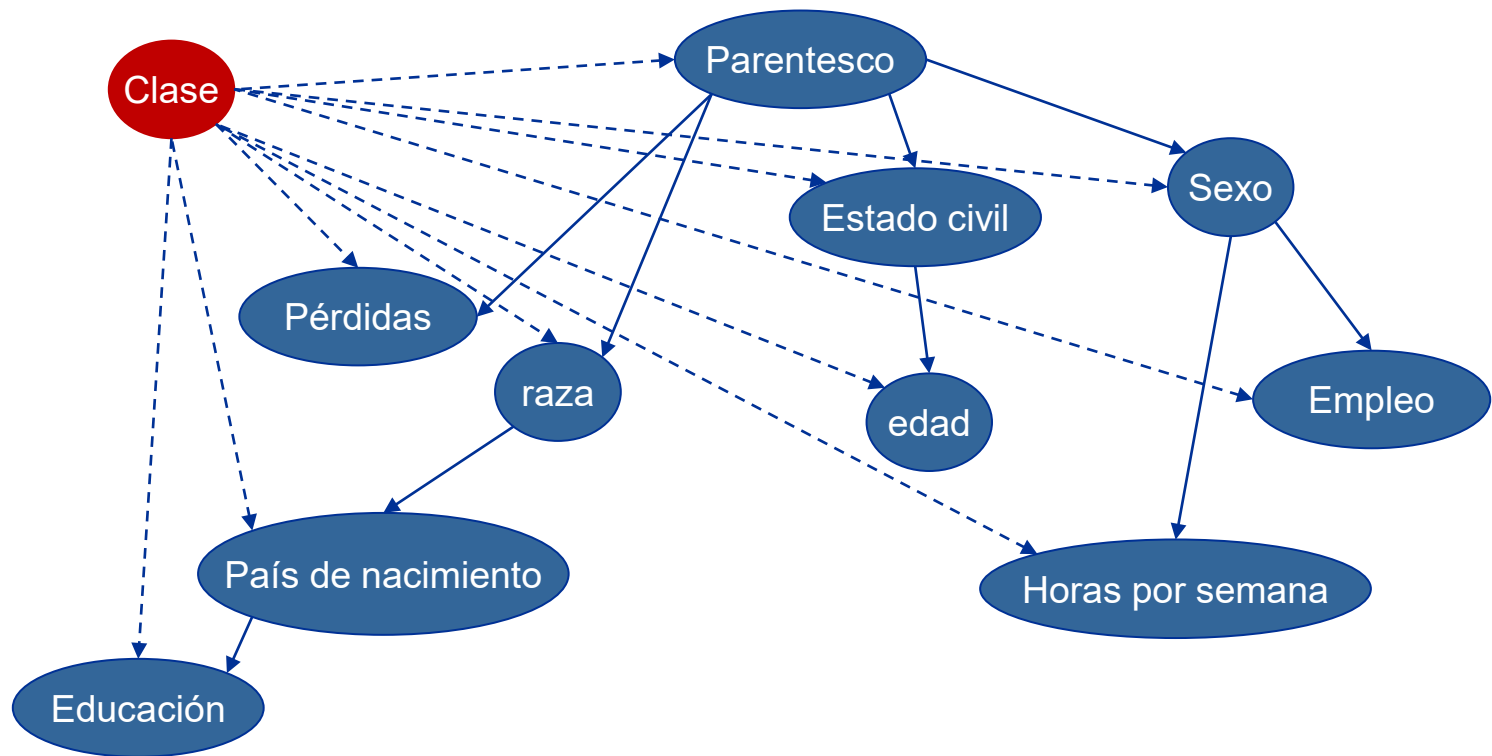
$$I_p(X, Y|C) = \sum_{x,y,c} p(x, y, c) \log \frac{P(x, y, c)}{P(x|c)P(y|c)}$$

- Después de construir la red con las relaciones entre los atributos siguiendo **una estructura de árbol** se añade la variable clase y se coloca como padre de todos ellos

Clasificación

Clasificadores bayesianos

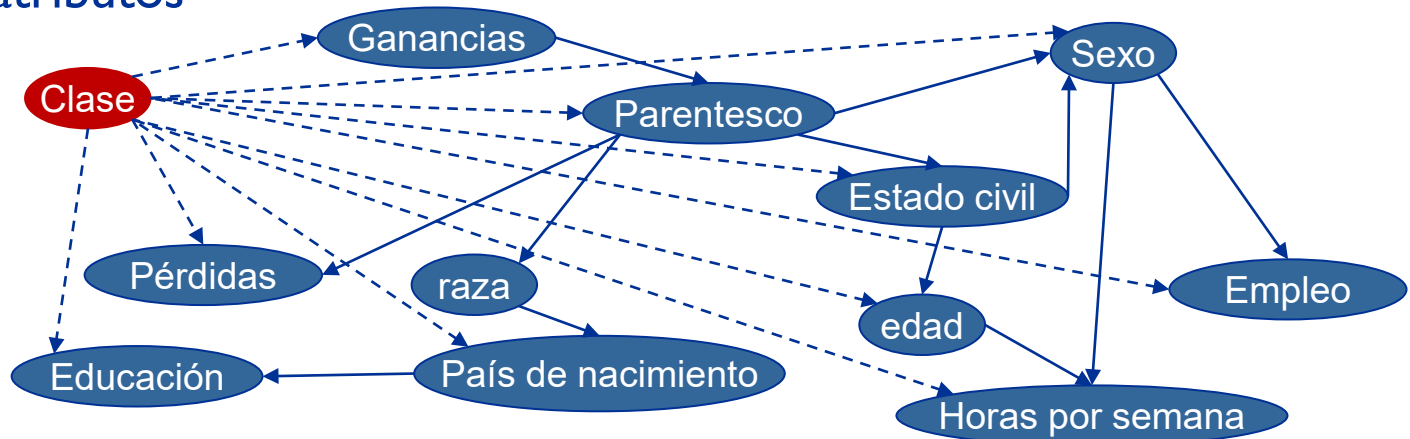
- **Algoritmo TAN** (*Tree Augmented Naïve Bayes*)



Clasificación

Clasificadores bayesianos

- **Algoritmo BAN** (*Bayesian Network Augmented Naïve Bayes*)
 - Se construye una red bayesiana para los atributos excluyendo la clase
 - La construcción de la red puede seguir un modelo BAN_{i-m} en el cual se parte de una estructura NB y se utiliza cualquier algoritmo de aprendizaje m para completar la red añadiendo arcos entre los atributos
 - Después de construir la red con las relaciones entre los atributos se añade la variable clase C y se añaden aristas desde C a todos los atributos



Clasificación

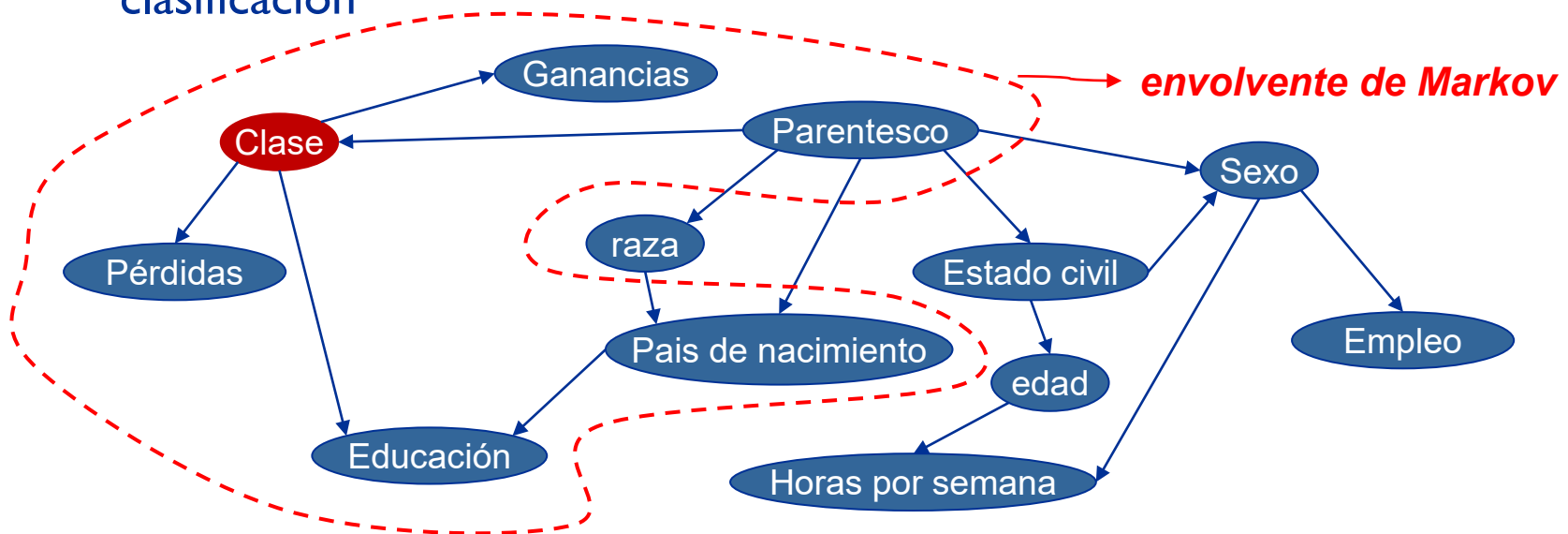
Clasificadores bayesianos

- **Envolvente de Markov**

- En una red bayesiana se cumple que toda variable X es independiente del resto dada su **envolvente de Markov** (*Markov blanket*) definida como:

$$(\text{padres}(X) \cup \text{hijos}(X) \cup \text{padres}(\text{hijos}(X)))$$

- Se seleccionan esas variables como variables predictoras útiles para la clasificación

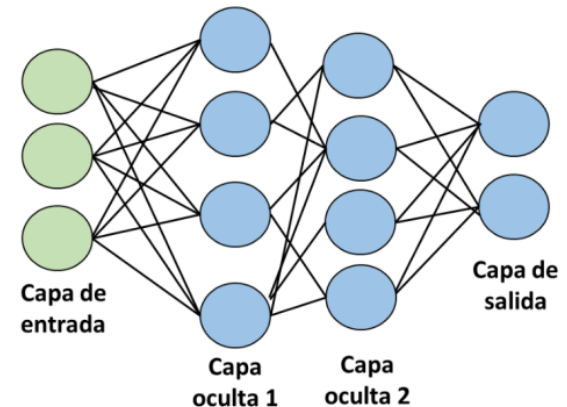


Clasificación

Inducción neuronal

Con esta técnica se representa el modelo como una arquitectura de nodos y enlaces ponderados que los conectan

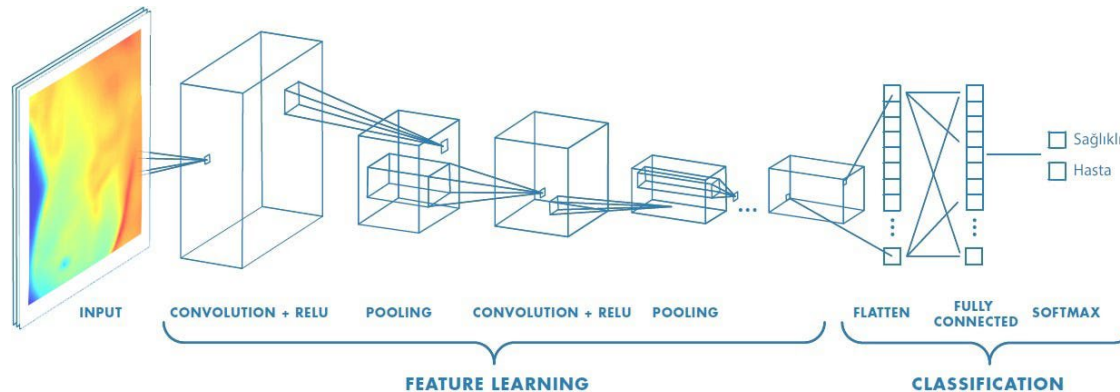
- Las redes neuronales son colecciones de nodos conectados con entrada, salida y procesamiento en cada nodo
- Entre las capas de entrada y salida visibles puede haber capas de procesamiento ocultas
- Cuando la red tiene muchas capas ocultas se llama *Deep Neural Network* (DNN).
- Algoritmos de Deep Learning (DL)
 - Deep Belief Networks (DBNs)
 - Convolutional Neural Networks (CNNs)
 - Recurrent Neural Networks (RNNs)
 - Generative Adversarial Networks (GANs)
 - Graph Neural Networks (GNNs)



Clasificación

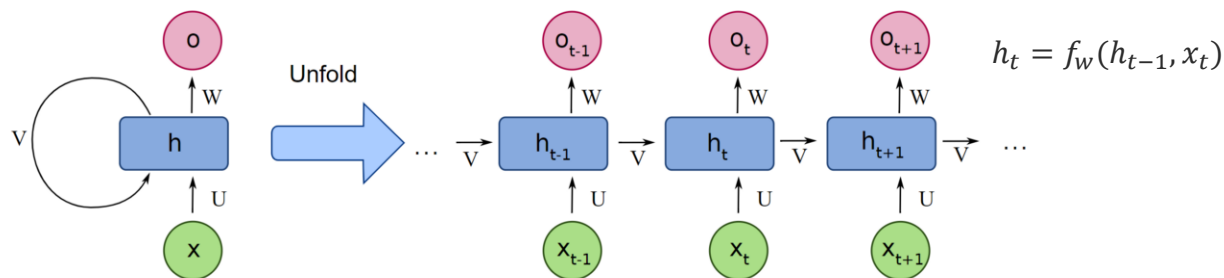
Inducción neuronal

- Convolutional Neural Network (CNN)



- Recurrent Neural Network (RNN)

- Long-Short Term Memory (LSTM)



Clasificación

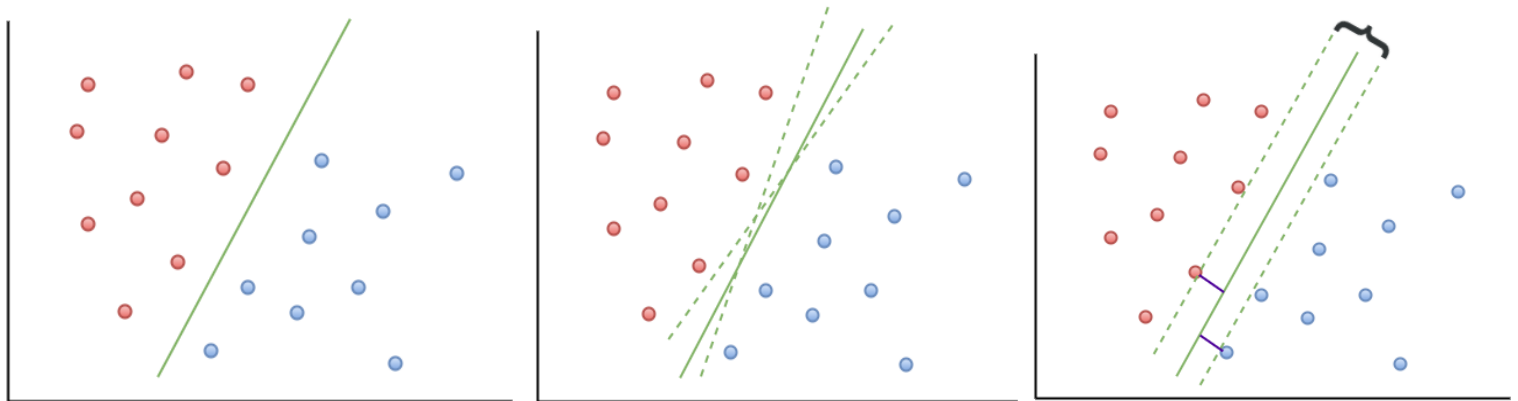
Inducción neuronal

- La inducción neuronal es una técnica más robusta que los árboles de decisión por la ponderación en los datos
- A veces la convergencia de las redes neuronales falla debido a impurezas o ruido en los datos de entrada o a la complejidad del problema
- La lógica interna que sigue el modelo para realizar la clasificación no es fácilmente accesible
- Existen dos maneras de solventar estas dificultades:
 - **Análisis de sensibilidad de las entradas:** permite determinar qué campos de entrada tienen más influencia en la imposición de las acciones del modelo clasificación
 - **Matrices de confusión:** ofrecen una medida del nivel de efectividad del modelo de clasificación, mostrando el número de clasificaciones correctas e incorrectas para cada posible valor de la variable que está siendo clasificada

Clasificación

Máquinas de Vectores de Soporte

- **SVM** (*Support Vector Machine*) es un clasificador lineal cuyo objetivo es encontrar el hiperplano que maximiza el margen entre los ejemplos de diferentes clases
 - Definido para clasificaciones binarias y atributos numéricos
 - Puede extenderse para tratar problemas con múltiples clases y atributos categóricos
 - Si los datos no son linealmente separables se pueden proyectar a un espacio de dimensión superior



Clasificación

Máquinas de Vectores de Soporte

- **Hiperplano**

Sea el conjunto de datos $D = \{(X_i, y_i)\}_{i=1}^n$ con n puntos en un espacio d -dimensional y donde $y_i \in \{+1, -1\}$

Función de discriminación lineal

$$h(X) = W^T X + b$$

$$h(X) = w_1 x_1 + \dots + w_d x_d + b$$

W es un vector de pesos d dimensional

b es un escalar (*bias*)

Los puntos del hiperplano cumplen

$$h(X) = W^T X + b = 0$$

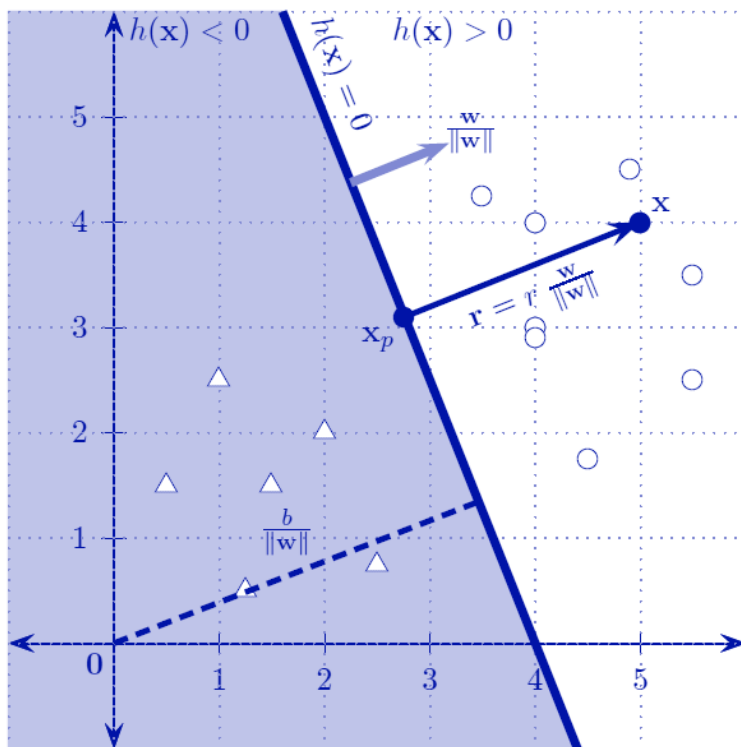
El hiperplano se define como el conjunto de puntos tales que $W^T X = -b$

El hiperplano $h(X)$ divide el espacio d -dimensional original en dos espacios

Clasificación

Máquinas de Vectores de Soporte

Si el conjunto de datos es linealmente separable se puede encontrar el hiperplano $h(X)$ tal que $y_i = +1 \Rightarrow h(x_i) > 0$ y $y_i = -1 \Rightarrow h(x_i) < 0$



$h(X)$ es un clasificador que permite predecir la clase y de cualquier ejemplo X

$$y = \begin{cases} +1 & \text{si } h(X) > 0 \\ -1 & \text{si } h(X) < 0 \end{cases}$$

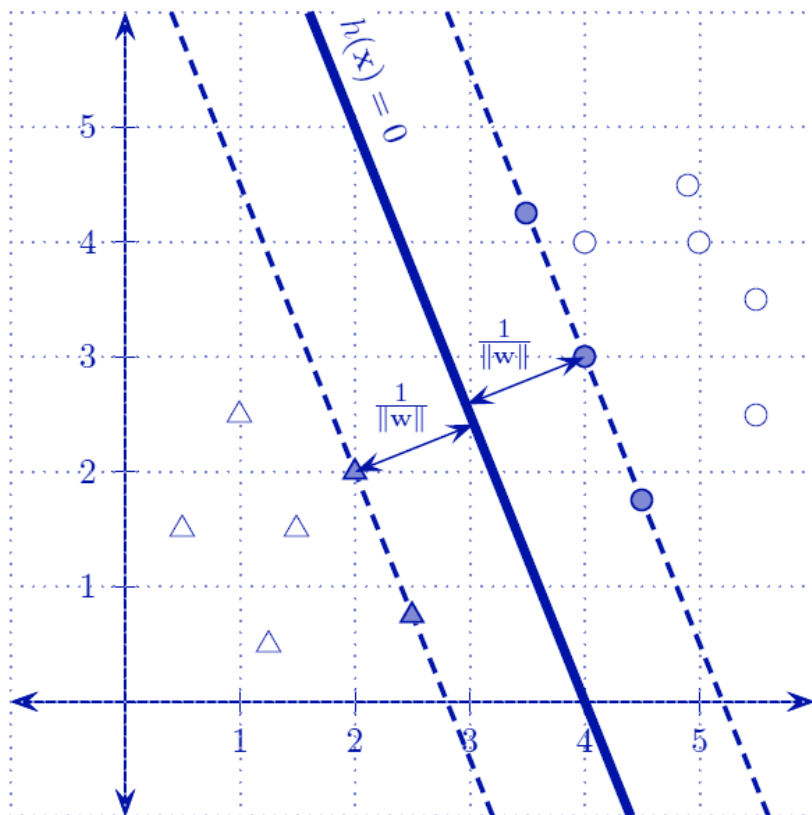
Distancia de cualquier punto x_i al hiperplano

$$\delta_i = \frac{y_i h(x_i)}{\|w\|} = \frac{y_i (w^T x_i + b)}{\|w\|}$$

Clasificación

Máquinas de Vectores de Soporte

- **SVM de margen máximo**



- **Margen** es la distancia al hiperplano del punto más cercano

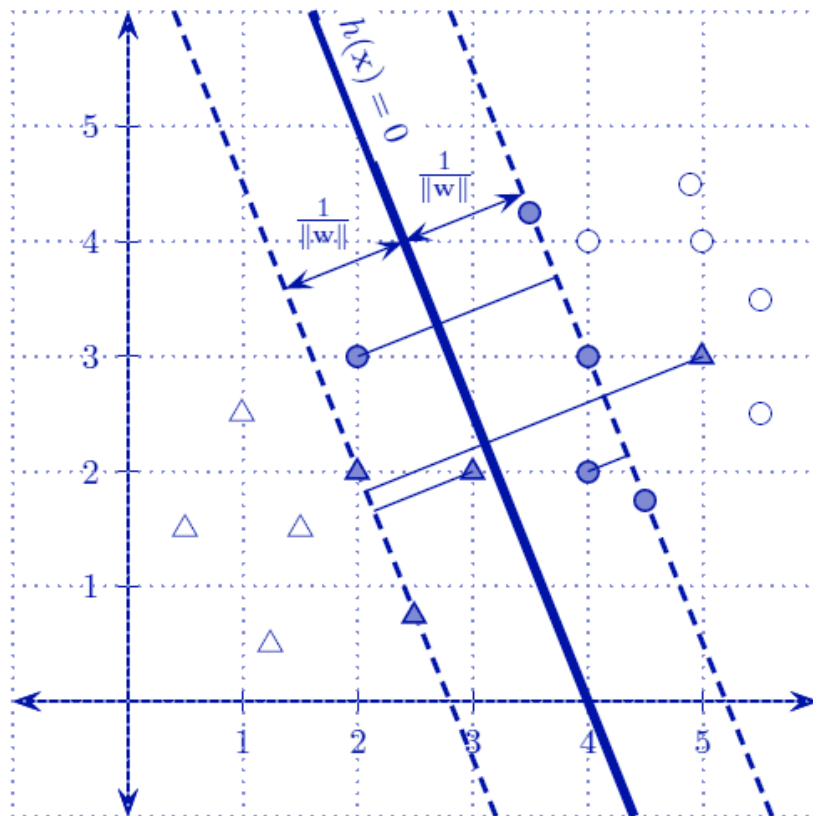
$$\delta^* = \min_{x_i} \left\{ \frac{y_i(w^T x_i + b)}{\|w\|} \right\}$$

- **Vectores de soporte** son aquellos puntos que se encuentran en el margen
- De todos los hiperplanos posibles, **el hiperplano óptimo será el que maximiza el margen**

Clasificación

Máquinas de Vectores de Soporte

- **SVM de margen blando**



- Si las clases no son totalmente separables se introduce una variable que indica el grado en que un punto incumple la regla de separabilidad

$$y_i (W^T X + b) \geq 1 - \xi_i$$

- Si $0 < \xi_i < 1$ el punto se encuentra entre el hiperplano y el margen
- Si $\xi_i \geq 1$ el punto se encuentra en lado contrario del hiperplano
- De todos los hiperplanos posibles, **el hiperplano óptimo será el que maximiza el margen y minimiza el término ξ_i**

Clasificación

Multclasificadores

Métodos que combinan hipótesis con el fin de mejorar la precisión de las predicciones

- Construyen un conjunto de hipótesis (*ensemble*) y combinan de alguna forma las predicciones del conjunto para clasificar ejemplos
- Se les conoce también con el nombre de métodos de construcción de metamodelos, modelos combinados o métodos de ensamblaje. Los más representativos son:
 - *Bagging* [Breiman, 1996]
 - *Boosting* [Freund y Schapire, 1996]
 - *Cross-validated committees* [Parmanto et al., 1996]
 - *Random forest* [Ho, 1998]

Clasificación

Multclasificadores

- Se dividen en dos grupos
 - **Métodos de ensamble o ensamblaje**
 - Conjuntos de modelos que se combinan para crear un nuevo modelo, empleando para ello la misma técnica de aprendizaje
 - **Métodos híbridos**
 - Combinaciones de algoritmos de aprendizaje que crean a su vez nuevas técnicas de aprendizaje híbridas
- Clasificaciones atendiendo a otras características:
 - Número de clasificadores individuales acoplados
 - Tipo de cada clasificador
 - Características de los subconjuntos de entrenamiento
 - Forma de combinar las hipótesis ...

Clasificación

Multclasificadores

- **Arquitecturas**

Dependiendo de la forma en que se desea integrar al conjunto para garantizar una toma de decisión se distinguen arquitecturas

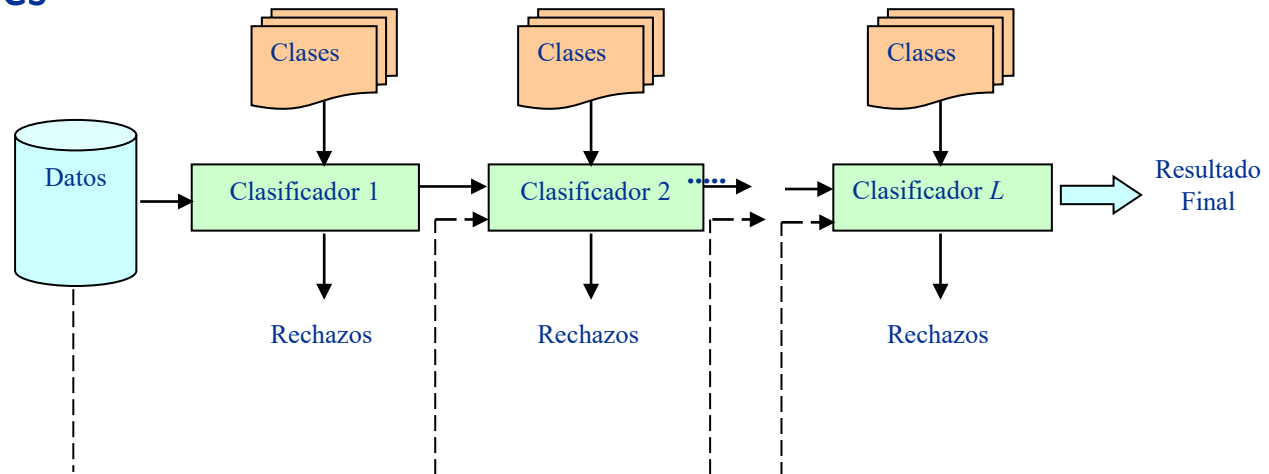
- **Secuencial o en serie:** la información resultante de un clasificador se convierte en información de entrada para otro
- **Paralela:** los clasificadores operan independientemente unos de otros, luego se fusionan sus respectivas respuestas y se busca un consenso entre los clasificadores para llegar a una única decisión
- **Híbrida:** se combinan las ventajas de las arquitecturas secuencial y paralela, es decir, la reducción del conjunto de las clases posibles y la búsqueda de un consenso entre los clasificadores

Clasificación

Multclasificadores

- **Arquitectura secuencial**

- La organización en niveles sucesivos de decisión permite reducir progresivamente el número de clases posibles
- En cada nivel: un único clasificador tiene en cuenta la respuesta proporcionada por el clasificador colocado anteriormente para tratar los rechazos y confirmar la decisión obtenida en el eslabón anterior.
- Existen dos enfoques: la re-evaluación y la reducción del conjunto de clases

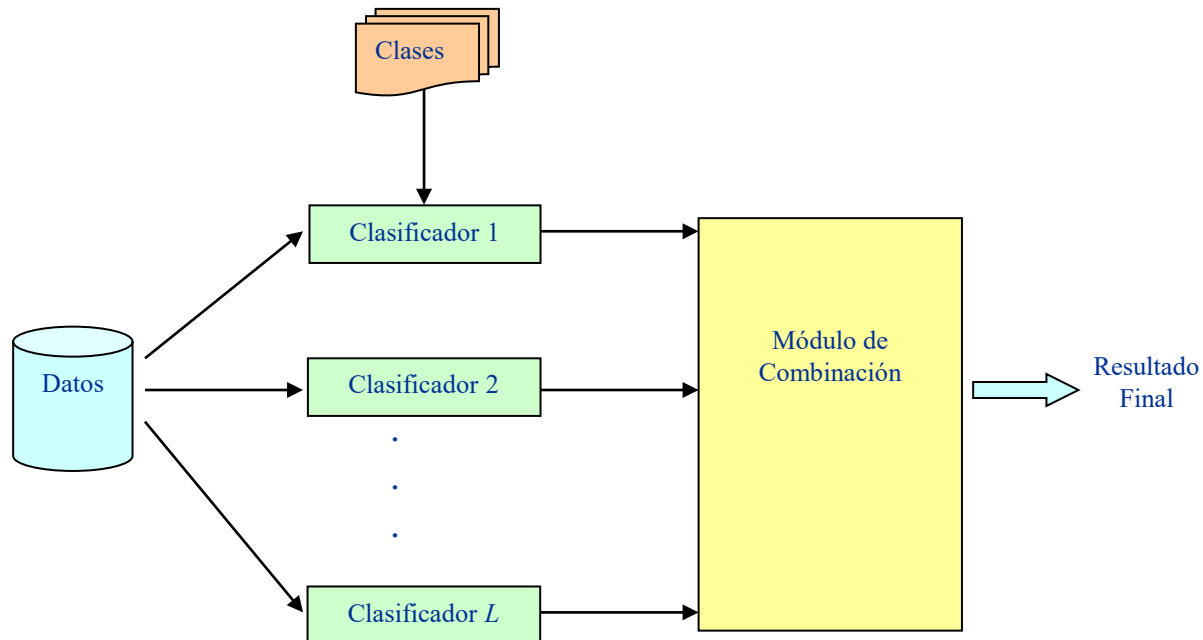


Clasificación

Multclasificadores

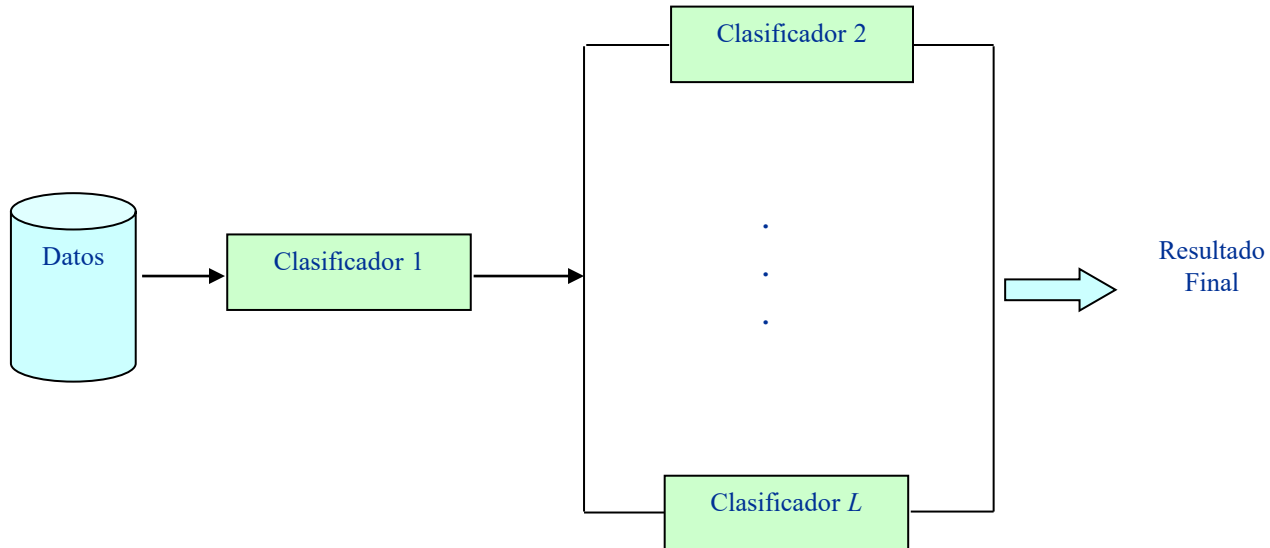
- **Arquitectura paralela**

- Los clasificadores trabajan de forma paralela proporcionando resultados que se pueden combinar mediante diferentes procedimientos para proporcionar el resultado final



Clasificación Multclasificadores

- **Arquitectura híbrida**
 - Combinación de las arquitecturas secuencial y paralela



Clasificación

Multclasificadores

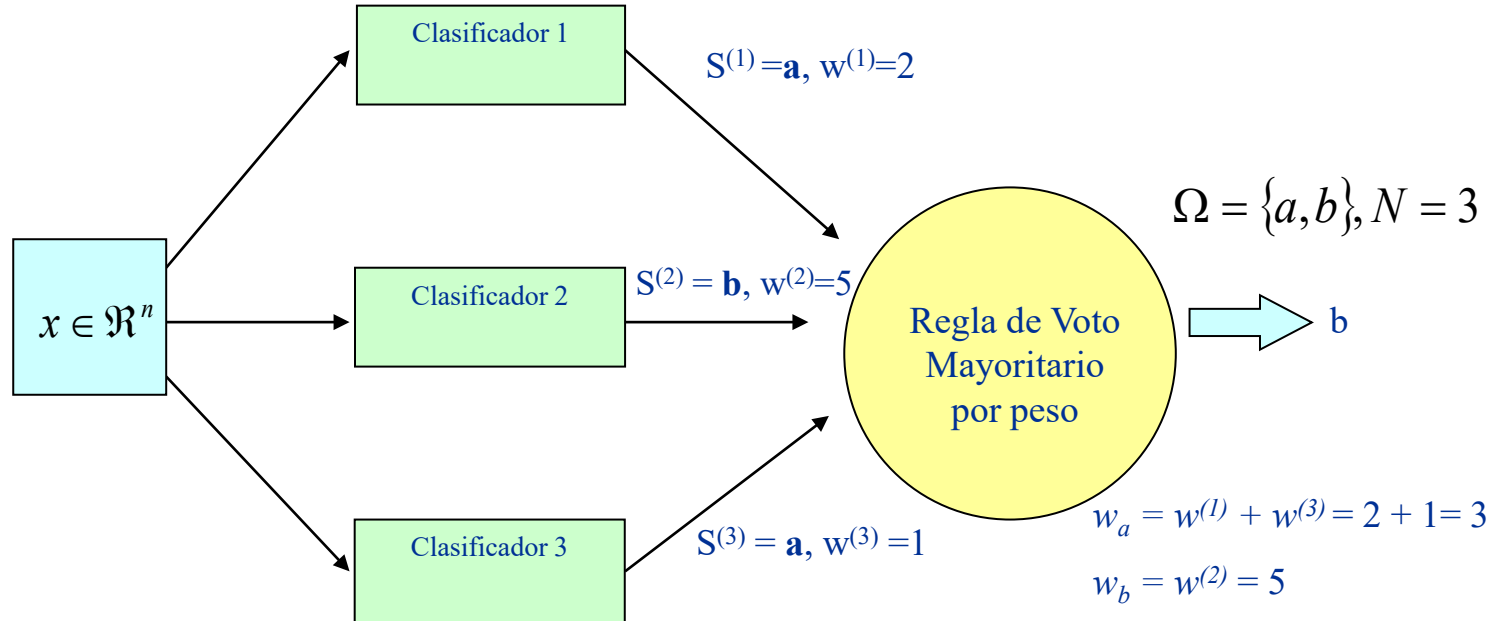
- **Estrategias de combinación (I)**

Formas en que se combinan las salidas de los clasificadores individuales

- **Voto mayoritario simple:** Cada clasificador tiene un voto que tiene igual valor. La clase con la mayoría de los votos es el resultado final
- **Voto mayoritario por peso:** consiste en entrenar a cada clasificador individualmente, y hacer que cada clasificador evalúe de manera individual unos datos de prueba fijados. El peso asignado al voto de cada clasificador se basa en la precisión obtenida en la prueba. La clase con mayor peso es la que se emite como resultado final del conjunto
- **Reglas basadas en el enfoque de Bayes:** Se utiliza la estadística bayesiana para determinar clasificaciones teóricamente óptimas del conjunto
- **Métodos de nivel de rango:** Un clasificador asocia frecuentemente a cada clase un “valor”, o una probabilidad, que indica su “grado de confianza” en la clasificación de un patrón x determinado. Esta información se puede utilizar para asignar un “rango” a cada clase. Las clases pueden ordenarse de forma que el mayor rango estará asociado a la clase con mayor probabilidad

Clasificación Multclasificadores

- Estrategias de combinación (II)



Ejemplo del método de voto mayoritario por peso

Clasificación

Multclasificadores

- **Métodos ensamble**

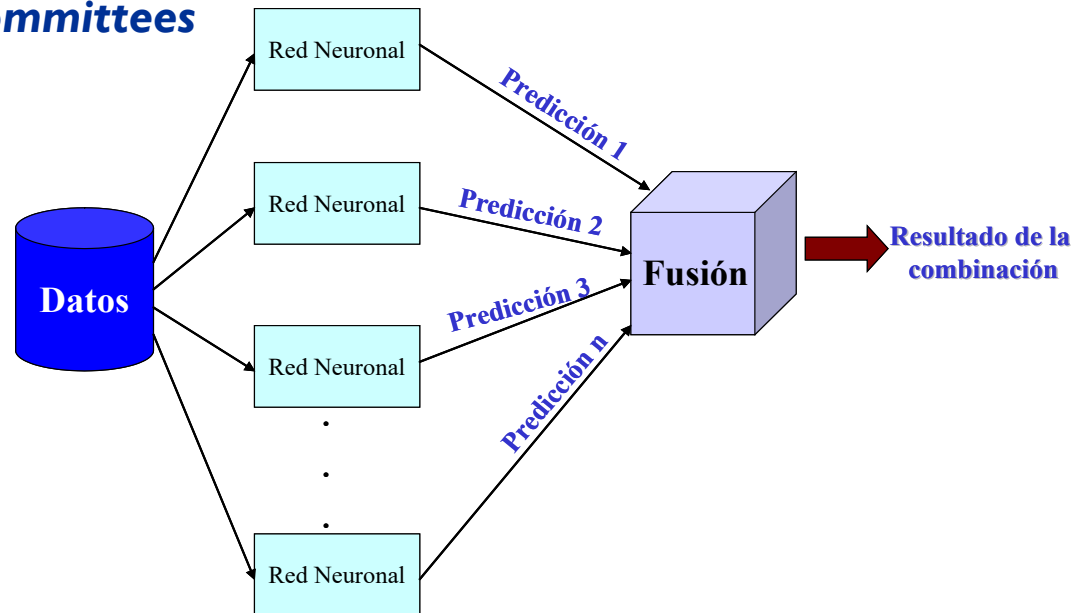
Se utilizan para resolver los siguientes problemas:

- **Problema estadístico:** se presenta cuando el algoritmo que aprende está buscando un espacio de las hipótesis H que es demasiado grande para la cantidad de datos de entrenamiento disponibles
- **Problema computacional:** en los casos donde existen suficientes datos de entrenamiento el problema puede ser aun difícil de resolver computacionalmente por el algoritmo de aprendizaje para encontrar la mejor hipótesis. Un ensamble construido mediante la búsqueda local a partir de diferentes puntos de partida puede proporcionar una mejor aproximación a la verdadera función desconocida que cualquiera de los clasificadores individuales.
- **Problema de representación:** se presenta cuando el espacio de hipótesis no contiene ninguna hipótesis que sea una buena aproximación a la función verdadera f , que aprende el algoritmo. En algunos casos, una suma de pesos de las hipótesis amplía el espacio de las funciones que pueden ser representadas

Clasificación Multclasificadores

- **Métodos ensamble**

- **Bagging** [Breiman, 1996]
- **Boosting** [Freund y Schapire, 1996]
- **Random forest** (*Random Subspace Method*) [Ho, 1988]
- **Error-Correcting Output Coding**
- **Cross-Validated Committees**
- **Randomization**



Clasificación

Multclasificadores

- **Bagging (Bootstrap AGGregatING)**
 - Basado en los conceptos de *bootstrapping* y agregación
 - *Bootstrapping* consiste en la generación de conjuntos de entrenamiento aleatorios con reemplazamiento (puede haber ejemplos repetidos)
 - Una muestra *bootstrap* de tamaño n se genera al seleccionar n instancias del conjunto de entrenamiento de manera aleatoria
 - De modo que se crean tantas muestras *bootstrap* del mismo tamaño como clasificadores existan
 - Cada clasificador se entrena con una réplica *bootstrap*
 - Para clasificar un ejemplo, se predice la clase de ese ejemplo para cada clasificador y se selecciona la clase con mayor voto

Clasificación

Multclasificadores

- **Boosting**

- Se basa en la asignación de un peso a cada ejemplo del conjunto de entrenamiento
 - En cada iteración, este método aprende un modelo que minimiza la suma de los pesos de los ejemplos clasificados erróneamente
 - Los errores de cada iteración se utilizan para actualizar los pesos de los ejemplos del conjunto de entrenamiento, de manera que se incremente el peso de los ejemplos errados y se reduce el peso de los ejemplos acertados
- **AdaBoost** es la variante de Boosting más conocida
 - En un ciclo se aprende un modelo a través de la evidencia ponderada, se estima el error del modelo y dependiendo del valor del error se detiene el algoritmo o se continua el proceso repitiendo el ciclo
 - Si se continúa iterando se actualiza los pesos de los ejemplos clasificados de forma acertada, se almacena el modelo y se efectúa la normalización del peso de todos los ejemplos

Clasificación

Multclasificadores

Algoritmo Adaboost

1. Se asigna el mismo peso a todos los ejemplos
2. Se aplica el algoritmo de aprendizaje
3. Se estima el error e del clasificador dividiendo la suma de los pesos de los ejemplos clasificados erróneamente entre la suma de los pesos de todos los ejemplos
4. Si el error no está entre 0 y 0.5 se detiene el algoritmo
5. Se actualiza el peso de los ejemplos clasificados correctamente multiplicándolo por $e/(1 - e)$
6. Se normalizan los pesos
7. Se vuelve al paso 2

Clasificación

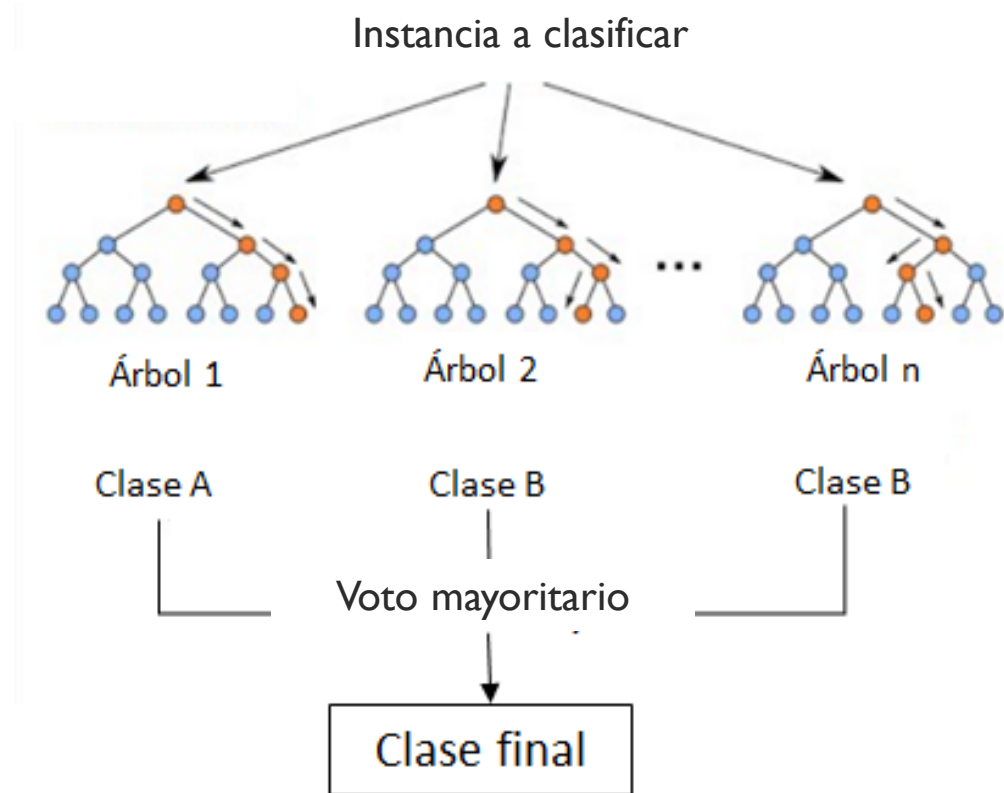
Multclasificadores

- **Random Forest**

- Es un *ensemble* de árboles de decisión que se entrenan a través del método de bagging
 - Todos los árboles realizan la predicción de la clase de una instancia dada
 - Se usa el voto mayoritario para seleccionar la clase que se dará como resultado
- Construcción de cada árbol
 - Entrenamiento del árbol con una selección aleatoria y con reemplazo de n instancias del conjunto de entrenamiento original
 - Selección aleatoria de un subconjunto de m variables. La mejor división de estos m atributos se usa para ramificar el árbol
 - No hay proceso de poda

Clasificación Multclasificadores

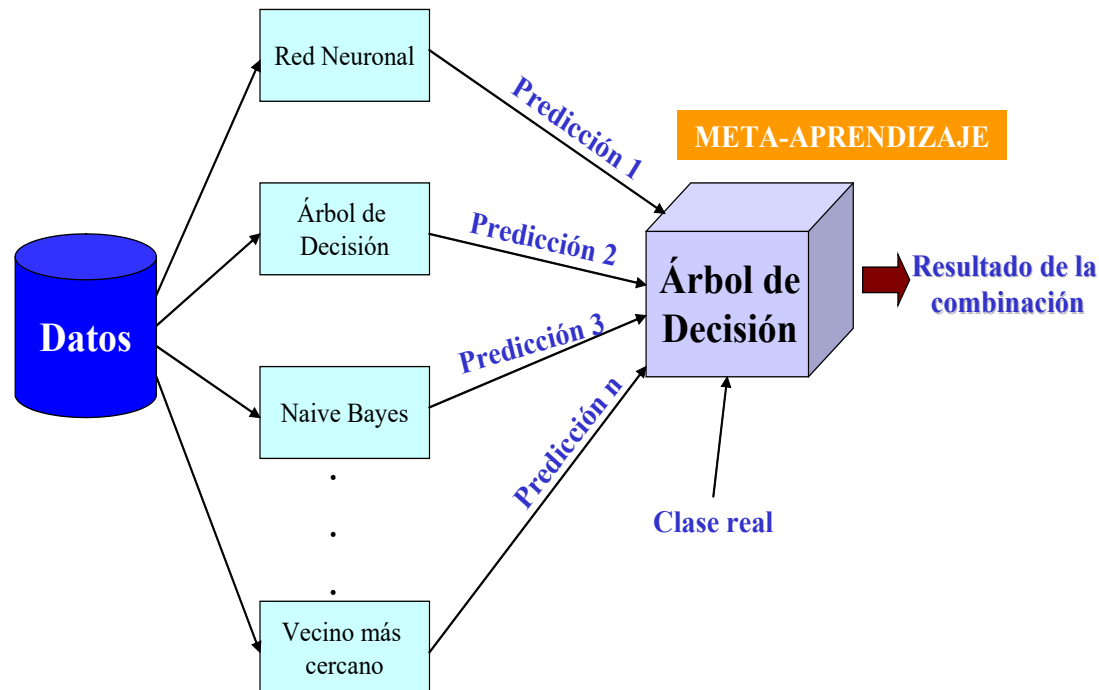
- **Random Forest**



Clasificación Multclasificadores

- **Métodos híbridos**

- Utilizan algoritmos de aprendizaje para aprender cómo integrar los resultados que proceden de distintos clasificadores (fase de meta-aprendizaje)
- **Métodos híbridos:** *stacking*, *cascading*, RBF (Radial Basis Functions)

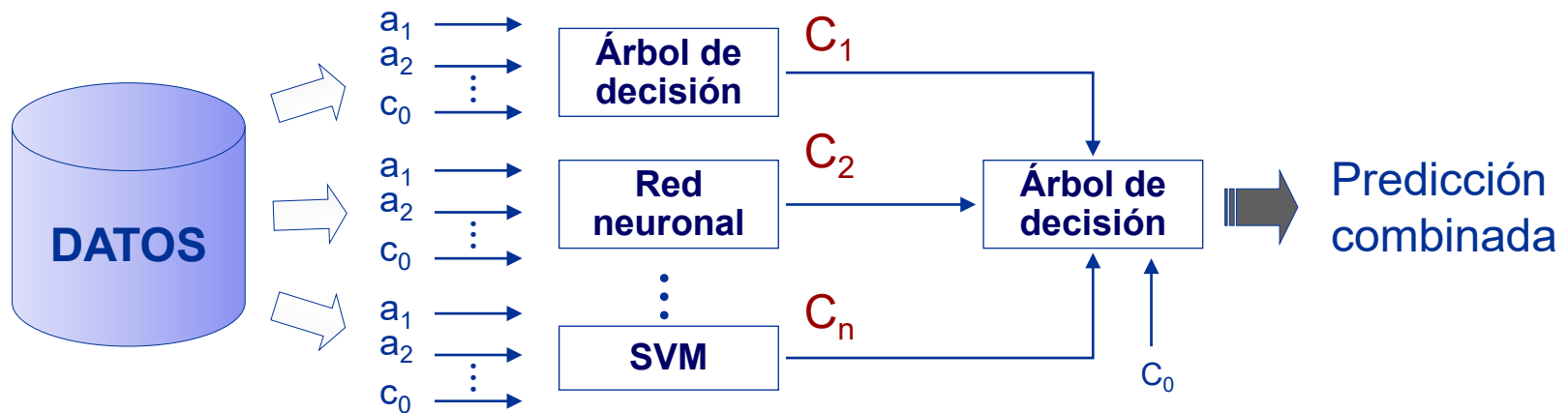


Clasificación

Multclasificadores

- **Stacking** [Wolpert, 1992]

- Combina múltiples clasificadores generados mediante distintos algoritmos de aprendizaje $L_1, \dots, L_N$ para un mismo conjunto de datos S , formado por ejemplos $s_i = (x_i, y_i)$.
 - En la primera fase, se genera un conjunto de clasificadores del nivel base $C_1, C_2, \dots, C_N$, donde $C_i = L_i(S)$.
 - En la segunda fase, un clasificador en el meta-nivel combina las salidas provenientes de los clasificadores del nivel base que trabajan en paralelo

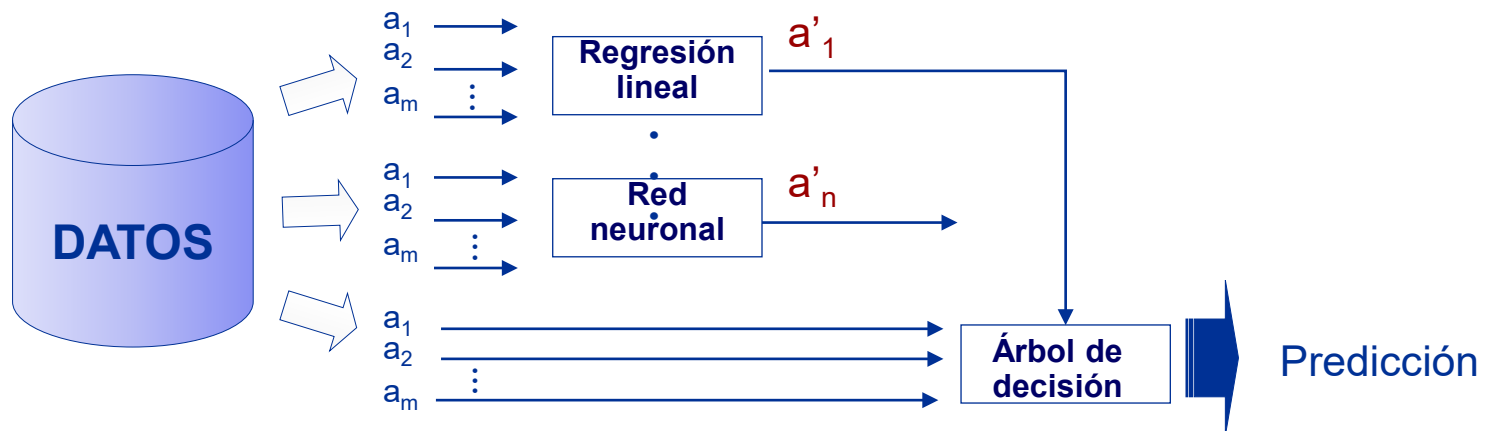


Clasificación

Multclasificadores

- **Cascading** [Gama y Brazdil, 2000]

- Permite mejorar los árboles de decisión
- La idea básica de la generalización de Cascading o *Cascade Generalization* es utilizar otros clasificadores para crear nuevos atributos
- En cada paso hay que realizar una extensión de los datos originales a través de la inserción de nuevos atributos



Clasificación

Clasificación multietiqueta

- Paradigma de clasificación que permite resolver problemas en los que los patrones tienen asociada más de una etiqueta

X_1	X_2	X_3	X_4	X_5	C
3.2	1.4	4.7	7.5	3.7	1
2.8	6.3	1.6	4.7	2.7	0
7.7	6.2	4.1	3.3	7.7	1
9.2	0.4	2.8	0.5	3.9	0
5.5	5.3	4.9	0.6	6.6	1

X_1	X_2	X_3	X_4	X_5	C_1	C_2	C_3	C_4
3.2	1.4	4.7	7.5	3.7	1	0	1	1
2.8	6.3	1.6	4.7	2.7	0	0	1	0
7.7	6.2	4.1	3.3	7.7	1	0	1	1
9.2	0.4	2.8	0.5	3.9	0	1	0	0
5.5	5.3	4.9	0.6	6.6	1	1	0	1

Clasificación

Clasificación multietiqueta

- **Clasificación de etiqueta única** (*single-label*)
 - Aprendizaje automático de un conjunto de casos que están asociados con una etiqueta de entre un conjunto de etiquetas L donde $|L| > 1$
- **Clasificación multietiqueta** (*multilabel*)
 - Los casos están asociados con un subconjunto de etiquetas Y , tales que $Y \subseteq L$

X_1	X_2	X_3	X_4	X_5	C_1	C_2	C_3	C_4	$Y \subseteq L$
3.2	1.4	4.7	7.5	3.7	1	0	0	1	$\{\lambda_1, \lambda_4\}$
2.8	6.3	1.6	4.7	2.7	0	0	1	1	$\{\lambda_3, \lambda_4\}$
7.7	6.2	4.1	3.3	7.7	1	0	0	1	$\{\lambda_1, \lambda_4\}$
9.2	0.4	2.8	0.5	3.9	0	1	0	0	$\{\lambda_2\}$
5.5	5.3	4.9	0.6	6.6	1	1	1	0	$\{\lambda_1, \lambda_2, \lambda_3\}$

Clasificación

Clasificación multietiqueta

- Tareas
 - **Clasificación multietiqueta**
 - Inducción de modelos que proporcionan la bipartición de etiquetas en relevantes e irrelevantes
 - **Ordenamiento de etiquetas** (*label ranking*)
 - Inducción de modelos que proporcionan las etiquetas para una instancia dada ordenadas según su relevancia
 - **Ordenamiento multi-etiqueta** (*multi-label ranking*)
 - Capacidad de extraer del conjunto de etiquetas tanto el ordenamiento como la bipartición

Clasificación

Clasificación multietiqueta

- La **clasificación** produce una bipartición del conjunto de etiquetas en relevantes e irrelevantes

- Dados los conjuntos

- $L = \{\lambda_j : j = 1 \dots q\}$ de etiquetas
- $D = \{(x_i, Y_i), i = 1 \dots m\}$ de instancias de entrenamiento

donde

- x_i es el vector de características y
- $Y_i \subset L$ es el subconjunto de etiquetas de la instancia i

- El subconjunto de etiquetas de la instancia i se define como un vector binario, $Y_i = \{y_1, y_2 \dots y_q\}$,

$y_j = 1$ indica la presencia de la etiqueta λ_j en el conjunto de etiquetas relevantes para x_i .

Clasificación

Clasificación multietiqueta

- Métodos

- De transformación

- Transforman la tarea de aprendizaje multietiqueta en una o más tareas de etiqueta simple
- Son independientes de los algoritmos
- Procedimientos
 - Relevancia binaria (***Binary relevance***)
 - Cadenas de clasificadores (***Classifier chains***)
 - Conjunto potencia de etiquetas (***Label powerset***)

- De adaptación de algoritmos

- Extienden algoritmos específicos para manejar datos multietiqueta directamente
- Boosting, SVM, árboles de decisión, redes neuronales, k-NN, clasificadores probabilísticos, redes bayesianas,...

Clasificación

Clasificación multietiqueta

- Ejemplo de transformación

x	$Y \subseteq L$
$x^{(1)}$	$\{\lambda_1, \lambda_4\}$
$x^{(2)}$	$\{\lambda_3, \lambda_4\}$
$x^{(3)}$	$\{\lambda_1, \lambda_4\}$
$x^{(4)}$	$\{\lambda_2\}$
$x^{(5)}$	$\{\lambda_1, \lambda_2, \lambda_3\}$

x	λ_1
$x^{(1)}$	true
$x^{(2)}$	false
$x^{(3)}$	true
$x^{(4)}$	false
$x^{(5)}$	true

x	λ_2
$x^{(1)}$	false
$x^{(2)}$	false
$x^{(3)}$	false
$x^{(4)}$	true
$x^{(5)}$	true

x	λ_3
$x^{(1)}$	false
$x^{(2)}$	true
$x^{(3)}$	false
$x^{(4)}$	false
$x^{(5)}$	true

x	λ_4
$x^{(1)}$	true
$x^{(2)}$	true
$x^{(3)}$	true
$x^{(4)}$	false
$x^{(5)}$	false

Clasificación

Clasificación multietiqueta

- Binary Relevance

X	y1	y2	y3	y4
X ⁽¹⁾	1	0	1	0
X ⁽²⁾	0	0	0	1
X ⁽³⁾	0	1	1	0
X ⁽⁴⁾	1	1	0	0

↔

X	y1
X ⁽¹⁾	1
X ⁽²⁾	0
X ⁽³⁾	0
X ⁽⁴⁾	1

X	y2
X ⁽¹⁾	0
X ⁽²⁾	0
X ⁽³⁾	1
X ⁽⁴⁾	1

X	y3
X ⁽¹⁾	1
X ⁽²⁾	0
X ⁽³⁾	1
X ⁽⁴⁾	0

X	y4
X ⁽¹⁾	0
X ⁽²⁾	1
X ⁽³⁾	0
X ⁽⁴⁾	0

- Classifier Chains

X	y1	y2	y3	y4
X ⁽¹⁾	1	0	1	0
X ⁽²⁾	0	0	0	1
X ⁽³⁾	0	1	1	0
X ⁽⁴⁾	1	1	0	0

↔

Clasificador 1	
X	y1
X ⁽¹⁾	1
X ⁽²⁾	0
X ⁽³⁾	0
X ⁽⁴⁾	1

Clasificador 2		
X	y1	y2
X ⁽¹⁾	1	0
X ⁽²⁾	0	0
X ⁽³⁾	0	1
X ⁽⁴⁾	1	1

Clasificador 3			
X	y1	y2	y3
X ⁽¹⁾	1	0	1
X ⁽²⁾	0	0	0
X ⁽³⁾	0	1	1
X ⁽⁴⁾	1	1	0

Clasificador 4				
X	y1	y2	y3	y4
X ⁽¹⁾	1	0	1	0
X ⁽²⁾	0	0	0	1
X ⁽³⁾	0	1	1	0
X ⁽⁴⁾	1	1	0	0

- Label Powerset

- Transforma cada combinación de etiquetas en una etiqueta única

Clasificación

Clasificación multietiqueta

- **Métricas para la evaluación de resultados (I)**
 - $L = \{\lambda_j : j = 1 \dots q\}$ conjunto de etiquetas
 - $D = \{ (x_i, Y_i), i = 1 \dots m \}$ conjunto de instancias multietiquetadas
 - x_i es el vector de características
 - $Y_i \subset L$ es el subconjunto de etiquetas de la instancia i
 - Z_i conjunto de etiquetas predichas para la instancia i
 - $r_i(\lambda)$ ranking predicho para una etiqueta λ
 - Las etiquetas más relevantes reciben la posición 1 mientras que las menos relevantes reciben una posición q

Clasificación

Clasificación multietiqueta

- **Métricas para la evaluación de resultados (II)**

- **Pérdida de Hamming:** promedio del número de veces que se clasificó erróneamente un par instancia-etiqueta

$$\text{Pérdida Hamming} = \frac{1}{m} \sum_{i=1}^m \frac{|Y_i \Delta Z_i|}{M}$$

m : número de instancias, M : número total de etiquetas

- **One-Error:** promedio del número de veces que la etiqueta con el mejor ranking predicho por el algoritmo de clasificación no está en el conjunto de etiquetas correctas de la instancia de entrada

$$\text{One - Error} = \frac{1}{m} \sum_{i=1}^m \delta (\arg \min r_i(\lambda))_{\lambda \in L} \quad \text{donde } \delta(\lambda) = \begin{cases} 1 & \text{si } \lambda \notin Y_i \\ 0 & \text{si } \lambda \in Y_i \end{cases}$$

Clasificación

Clasificación multietiqueta

- **Métricas para la evaluación de resultados (IV)**

- **Precisión:** proporción de etiquetas relevantes dentro del conjunto de etiquetas predichas
- **Recall:** proporción de etiquetas relevantes de la instancia que son correctamente predichas
- **F_1 :** media armónica de precisión y recall

$$F_1 = \frac{1}{m} \sum_{i=1}^m \frac{2|Y_i \cap Z_i|}{|Y_i| + |Z_i|}$$

- **Precisión promedio:** proporción media de las etiquetas clasificadas correctamente por encima de una etiqueta en particular

$$\text{Precisión promedio} = \frac{1}{m} \sum_{i=1}^m \frac{1}{|Y_i|} \sum_{\lambda \in Y_i} \frac{|\{\lambda' \in Y_i: r_i(\lambda') \leq r_i(\lambda)\}|}{r_i(\lambda)}$$

Clasificación

Clasificación multietiqueta

- **Métricas para la evaluación de resultados (III)**

- **Cobertura (coverage)**: indica hasta qué punto, en promedio, es necesario ir abajo en la lista de etiquetas ordenadas por rango a fin de incluir todas las etiquetas que deberían haber sido asignadas a la instancia de entrada

$$\text{Cobertura} = \frac{1}{m} \sum_{i=1}^m \max_{\lambda \in Y_i} r_i(\lambda) - 1$$

- **Ranking loss**: promedio del número de veces que una etiqueta irrelevante se le asigna un ranking mejor que el de etiquetas relevantes

$$\text{Ranking loss} = \frac{1}{m} \sum_{i=1}^m \frac{1}{|Y_i| |\bar{Y}_i|} |\{(\lambda_a, \lambda_b) : r_i(\lambda_a) > r_i(\lambda_b), (\lambda_a, \lambda_b) \in Y_i \times \bar{Y}_i\}|$$

3. MÉTODOS DE SELECCIÓN DE CARACTERÍSTICAS

Métodos de selección de características

- Reducción del número de atributos en problemas de alta dimensionalidad
 - Minería de textos
 - Análisis de expresión génica
 - ...
- Objetivos
 - Mejorar la precisión de las predicciones
 - Aumentar el rendimiento de los clasificadores
 - Conseguir una mejor comprensión del proceso subyacente

Métodos de selección de características

Métodos basados en la ganancia de información

- Evalúan la importancia de un atributo en función de la ganancia de información que proporciona con respecto a la clase
 - **Ganancia de información** (GI)

$$GI (Clase, Atributo) = H(Clase) - H(Clase, Atributo)$$

$H(Clase)$: entropía o cantidad esperada de información de la clase

$H(Clase, Atributo)$: entropía o cantidad esperada de información de la clase dado el atributo

$$H = \sum_j -p_j \log_2 p_j$$

p_j probabilidad de la clase j

- Se evalúan los valores de ganancia de cada uno de los atributos y se ordenan por dicho valor

Métodos de selección de características

CFS (*Correlation-based Feature Subset Selection*)

- Evalúa la importancia de un subconjunto de atributos o características teniendo en cuenta su capacidad individual de predicción de la clase junto con el grado de redundancia entre ellos
- Función de evaluación

$$M_S = \frac{k \overline{r_{cf}}}{\sqrt{k + k(k-1) \overline{r_{ff}}}}$$

M_S representa el "mérito" o relevancia de un subconjunto S que contiene k características

$\overline{r_{cf}}$ es la correlación media clase-característica ($f \in S$)

$\overline{r_{ff}}$ es la intercorrelación media característica-característica

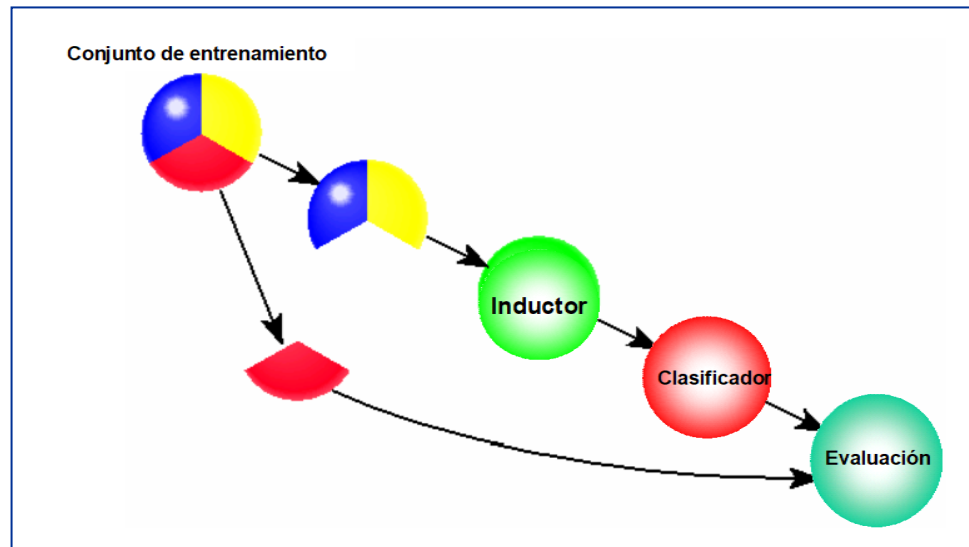
- El numerador representa el poder de predicción de un conjunto de características
- El denominador representa el grado de redundancia entre dichas características

4. EVALUACIÓN DE LOS CLASIFICADORES

Evaluación de los clasificadores

Estimación de errores en los clasificadores

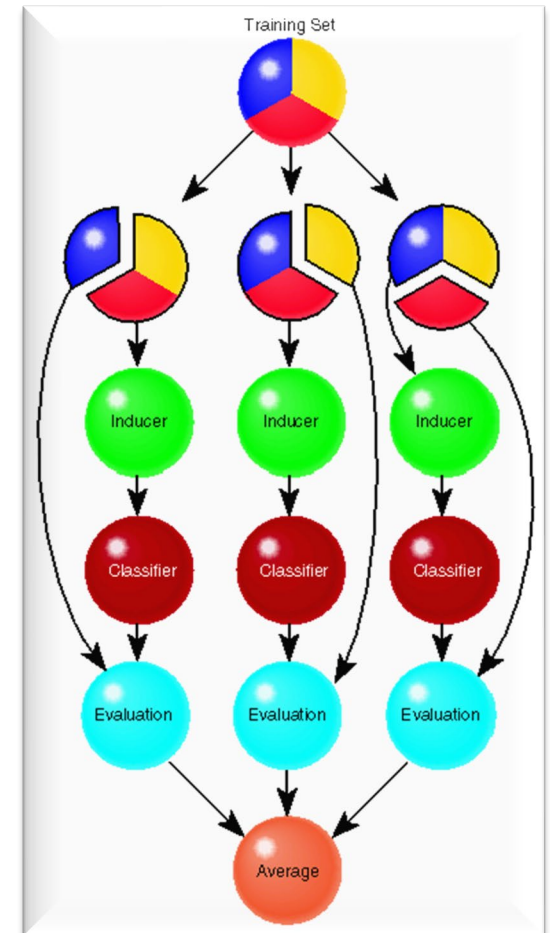
- Cuando se construye el clasificador se puede predecir cual será su índice de error cuando se aplique a datos sin clasificar
- **Técnica “Holdout”**: consiste en utilizar una porción de registros como conjunto de entrenamiento y el resto como conjunto de prueba



Evaluación de los clasificadores

Estimación de errores en los clasificadores

- **Validación cruzada:** Los datos se dividen en k subconjuntos mutuamente excluyentes del mismo tamaño aproximadamente
 - Se realizan k entrenamientos de la forma que muestra la figura
 - El índice de error estimado es la media de los errores obtenidos
 - La validación cruzada se puede repetir t veces. En ese caso se construyen y evalúan $k * t$ clasificadores

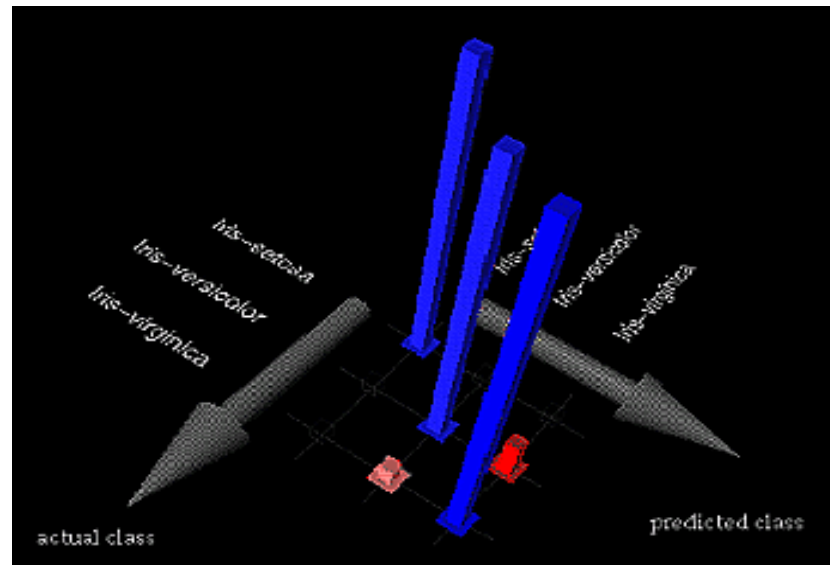


Evaluación de los clasificadores

Matrices de confusión

- Muestran el tipo de las predicciones correctas e incorrectas

	Real	
	True Positives (TP)	False Positives (FP)
Estimado	False Negatives (FN)	True Negatives (TN)



Evaluación de los clasificadores

Matrices de confusión

- Permiten calcular métricas para cada una de las clases
 - Precisión
 - Recall...

		Predicho	
		Abandonarán	Permanecerán
Real	Han abandonado	402	198
	Han permanecido	62	7598
		↓ 464	↓ 7796

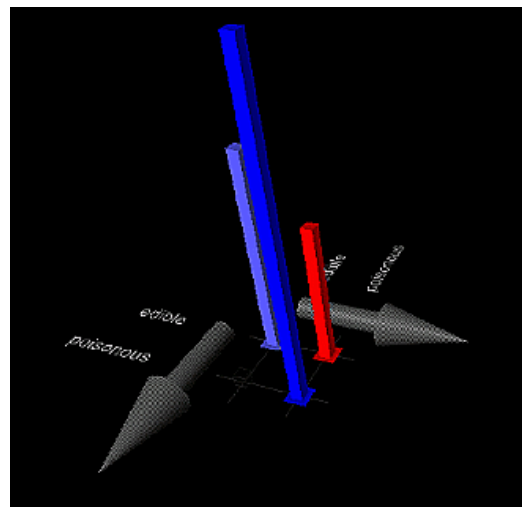
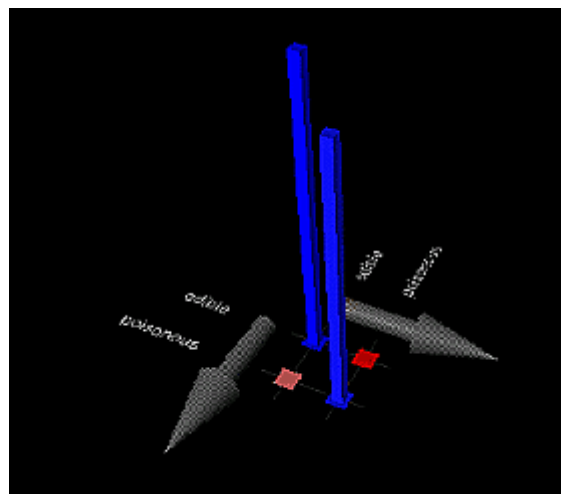
Errores

$$\text{precisión (abandonan)} = 402/464 = 0.87$$

Evaluación de los clasificadores

Matrices de pérdida

- Matrices utilizadas para construir matrices de confusión en las que se considera el coste asociado a las predicciones incorrectas
- La minimización del coste se puede realizar de dos formas:
 - Reduciendo al máximo las predicciones incorrectas de mayor coste y aumentando las de menor coste
 - Introduciendo una nueva clase para las predicciones indeterminadas (probabilidad de acierto muy baja)



Evaluación de los clasificadores

Métricas

- **Exactitud** (*accuracy*)
 - Porcentaje de instancias correctamente clasificadas
- **Precisión**
 - Probabilidad de que un ejemplo sea positivo si el clasificador lo clasifica como positivo

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- **Recall o sensibilidad**
 - Probabilidad de que un ejemplo positivo sea clasificado como positivo

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

	Real	
Estimado	True Positives (TP)	False Positives (FP)
	False Negatives (FN)	True Negatives (TN)

Evaluación de los clasificadores

Métricas

- **Kappa de Cohen**

- Probabilidad de que una instancia se clasifique exactamente igual en dos ejecuciones paralelas del algoritmo

$$K = \frac{P(\text{concordancia}) - P(\text{azar})}{1 - P(\text{azar})}$$

$P(\text{concordancia})$: probabilidad de concordancia real

$P(\text{azar})$: probabilidad de concordancia por azar

- El valor máximo de kappa se alcanza cuando hay coincidencia total
- Inconvenientes
 - Kappa es dependiente del número de clases
 - El valor de kappa se ve afectado por la prevalencia de la característica estudiada

Evaluación de los clasificadores

Métricas

- **Kappa de Cohen**

- Ejemplo para dos ejecuciones (A y B) de un algoritmo

$$K = \frac{P(\text{concordancia}) - P(\text{azar})}{1 - P(\text{azar})}$$

$$P(\text{concordancia}) = \frac{\text{conc. SI} + \text{conc. NO}}{n^{\circ} \text{ de ejemplos}}$$

$$P(\text{azar}) = P(\text{SI}) + P(\text{NO})$$

$$P(\text{SI}) = P(\text{SI}|A) * P(\text{SI}|B)$$

$$P(\text{NO}) = P(\text{NO}|A) * P(\text{NO}|B)$$

		B	
		SI	NO
A	SI	4	6
	NO	10	80

Evaluación de los clasificadores

Análisis ROC (*Receiver Operating Characteristic*)

- Cuando la matriz de coste no se conoce durante el aprendizaje se crean un conjunto de clasificadores y se selecciona el que mejor se comporte para contextos de coste determinados a posteriori
 - Se utiliza para problemas de dos clases: **positiva** y **negativa**
 - **Matriz de confusión:**

	Real	
	True Positives (TP)	False Positives (FP)
Estimado	False Negatives (FN)	True Negatives (TN)

- **Valores normalizados:**

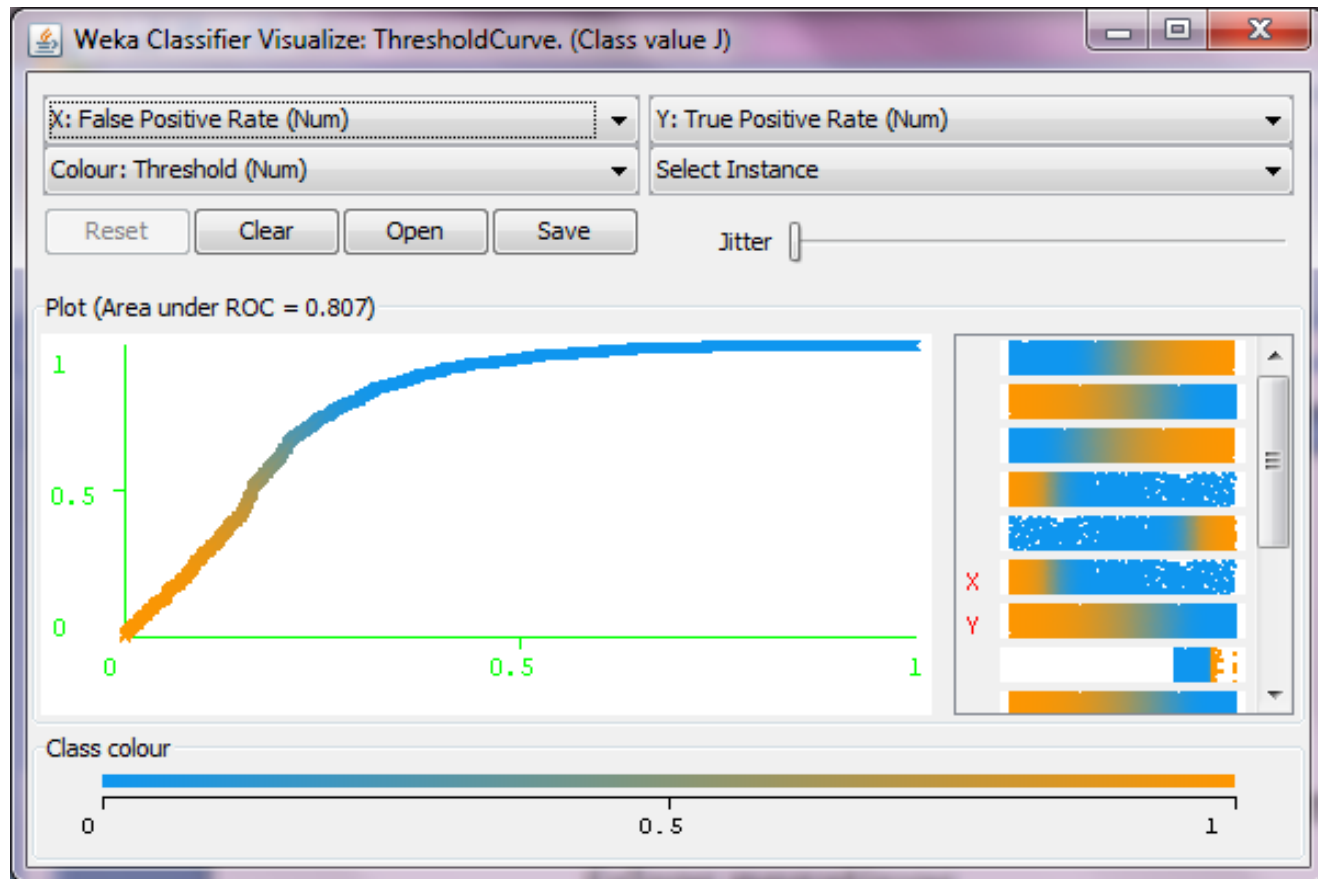
- True Positive Rate: $TPR = TP / (TP + FN)$
- False Positive Rate: $FPR = FP / (FP + TN)$
- False Negative Rate: $FNR = FN / (TP + FN)$
- True Negative Rate: $TNR = TN / (FP + TN)$

	Real	
	TPR	FPR
Estimado	FNR	TNR

Matriz de confusión normalizada

Evaluación de los clasificadores

Análisis ROC (Receiver Operating Characteristic)

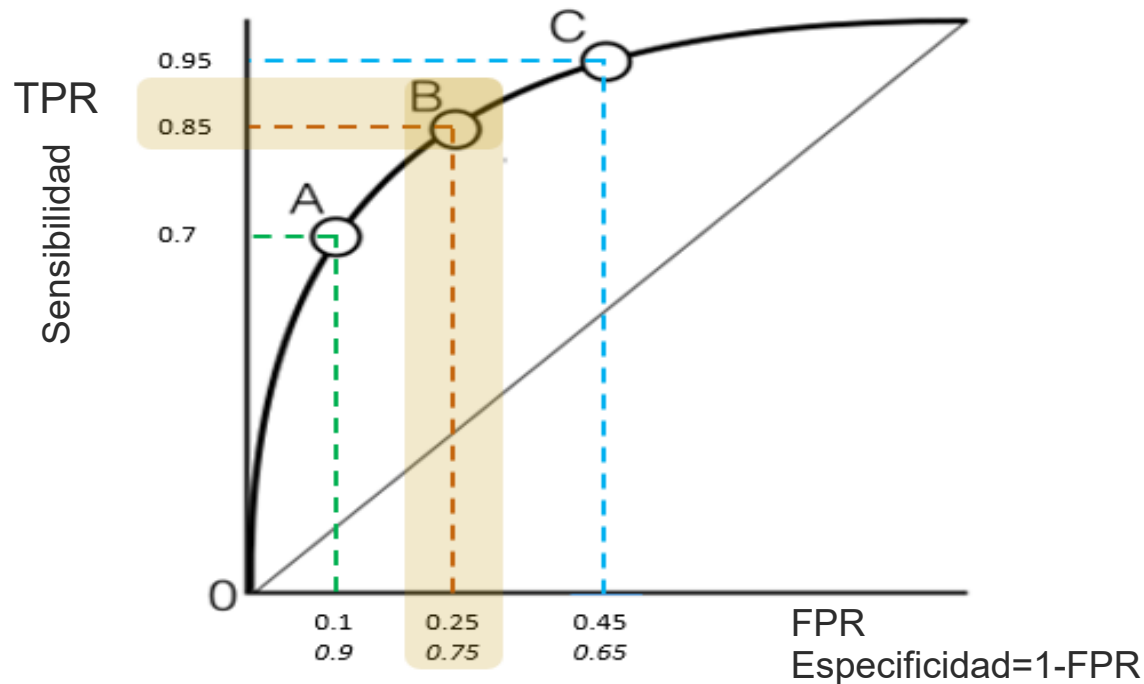


Clasificador en el espacio ROC

Evaluación de los clasificadores

Análisis ROC (Receiver Operating Characteristic)

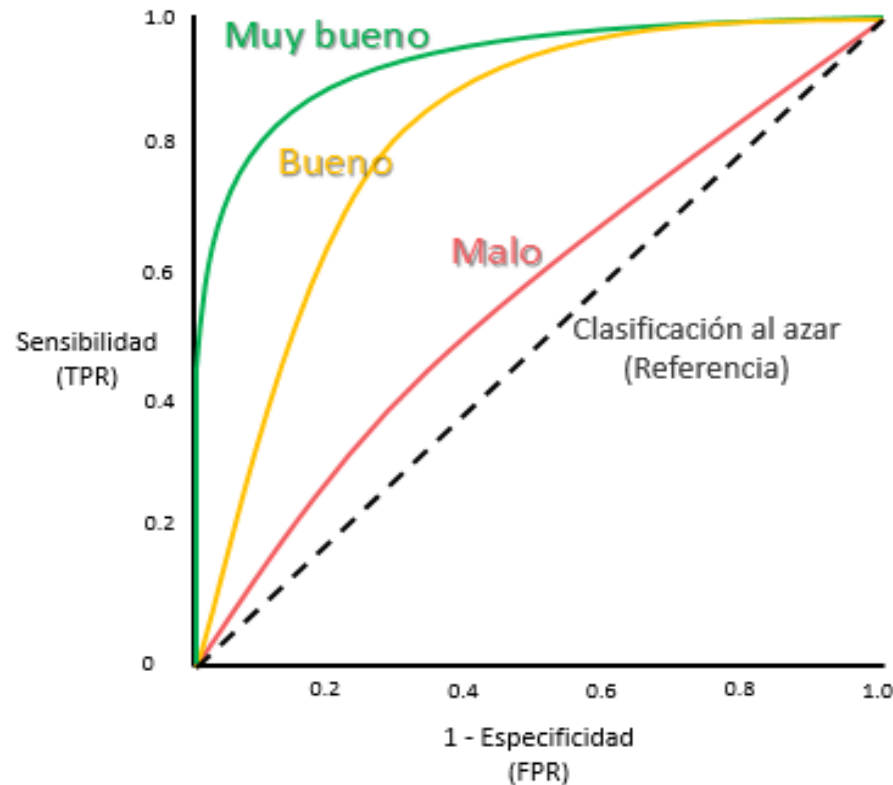
- **AUC (Area Under ROC Curve)**
 - El área bajo la curva ROC puede utilizarse como una medida de evaluación de los clasificadores



Evaluación de los clasificadores

Análisis ROC (Receiver Operating Characteristic)

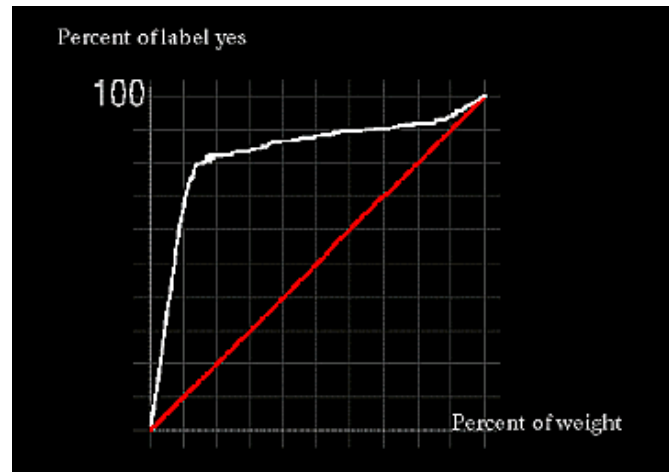
- **AUC (Area Under ROC Curve)**
 - Cuanto mayor sea el área bajo la curva ROC mejor será el clasificador



Evaluación de los clasificadores

Curvas de eficacia (lift curves)

- Muestran el peso acumulado de los registros con un valor específico de la etiqueta en relación con el peso de todos los registros
- La curva muestra la diferencia entre la ordenación de los registros al azar (curva roja) y ordenados según las predicciones del clasificador (curva blanca)



Evaluación de los clasificadores

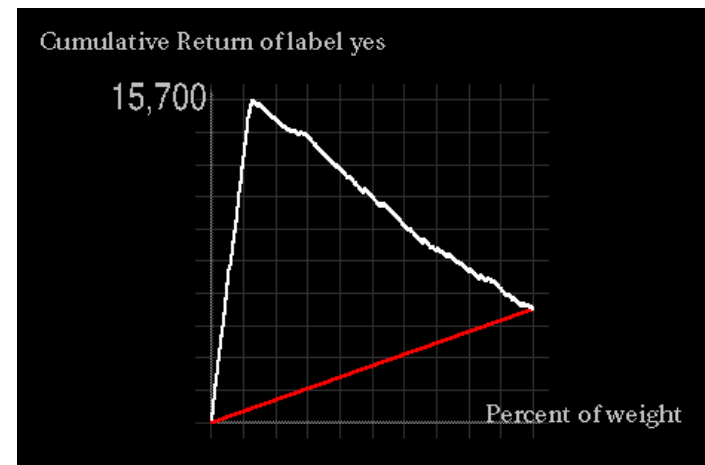
Curvas ROI (Return-On-Investment)

- Similares a las curvas de esfuerzo, pero muestran la precisión en función de la pérdida, en lugar de hacerlo en función del error
 - Los puntos de la curva se ordenan por la pérdida esperada para cada registro
 - La altura de cada punto indica el beneficio acumulado (pérdida inversa) de los registros

Loss matrix for churned

Actual Values	Predicted Values		
	?	no	yes
no	100000	0	10
yes	100000	100	-90

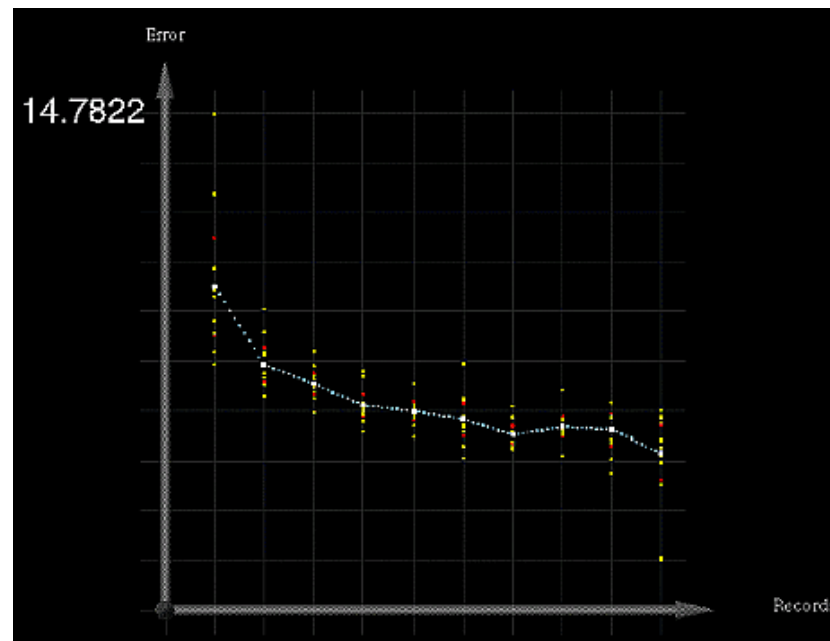
OK Reset Cancel



Evaluación de los clasificadores

Curvas de aprendizaje

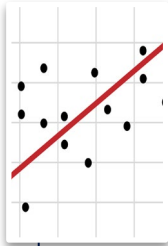
- Gráfico que muestra el error de un clasificador en función del número de registros usados para crearlo
- Cada clasificador se genera usando una muestra de registros al azar, y su error se estima usando el resto de los registros



5. PREDICCIÓN DE VALORES

Predicción de valores

Técnicas



Regresión

- lineal
- no lineal
- Función base radial



Series temporales

- Las técnicas tradicionales de predicción de valores son la **regresión lineal y no lineal**, sin embargo estas técnicas presentan bastantes deficiencias
- Una nueva técnica de regresión, más robusta, es la conocida como **función base radial**
- En problemas con características dependientes del tiempo se puede hacer uso de las **series temporales** para la predicción

Predicción de valores

Regresión

Regresión lineal

- Ajuste de un conjunto de datos a una línea recta

Vector n -dimensional de entradas
(variables independientes)

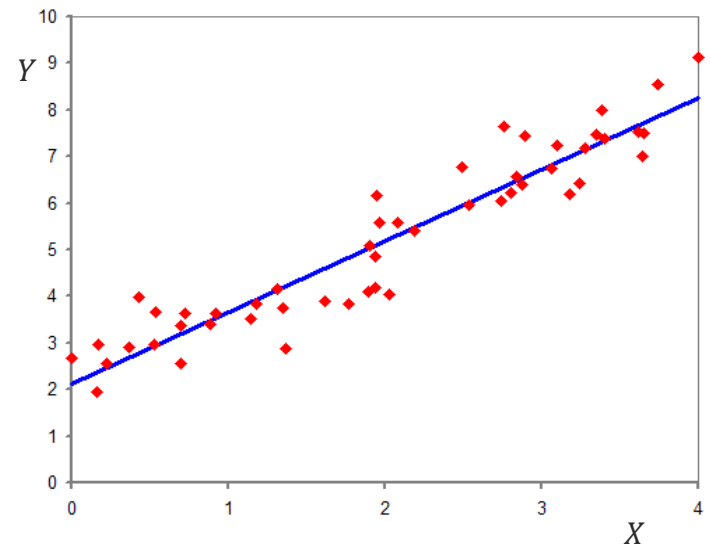
$$X = [x_1, x_2, \dots, x_n]$$

Vector de $n + 1$ parámetros

$$\Theta = \theta_0, \theta_1, \dots, \theta_n$$

Variable dependiente $Y = f(X|\Theta)$

$$f(X|\Theta) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$



Predicción de valores

Regresión

$$Y = \theta_0 + \theta_1 x_1$$

Regresión lineal simple

- Sólo una variable independiente

$$Y = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

Regresión lineal múltiple

- Más de una variable independientes

- Obtención del valor de los parámetros Θ

- **Método de mínimos cuadrados**

Función de pérdida : error cuadrático medio

$$L(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2 = \frac{1}{n} \sum_{i=1}^n (\langle \Theta, x_i \rangle - y_i)^2$$

Se aplica una técnica de optimización para encontrar el valor de los parámetros que minimice la función de pérdida

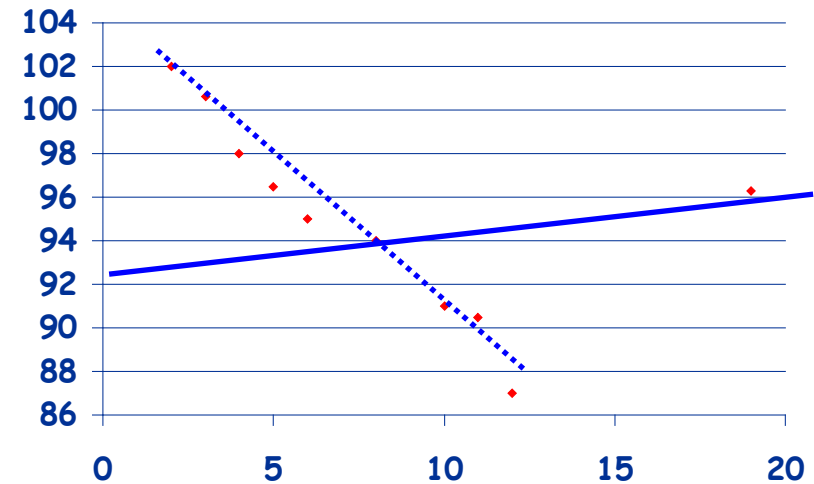
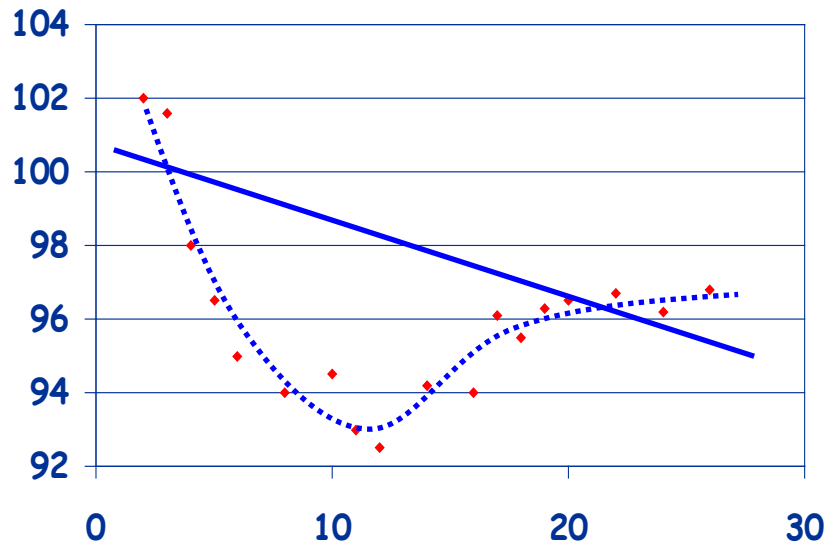
Predicción de valores

Regresión

- Problemas de la regresión lineal
 - Intentar ajustar datos con comportamiento no lineal a una línea recta
 - Sensibilidad a los datos con ruido
 - Al intentar encontrar los parámetros de la ecuación que minimicen la distancia de la recta a todos los puntos podemos obtener una recta que no refleja realmente el comportamiento de los datos
- Si el comportamiento de los datos no es lineal, se pueden añadir términos **no lineales** a las ecuaciones de regresión lineal, pero la técnica es costosa y no es muy robusta

Predicción de valores

Regresión



— Línea de regresión predicha
..... Línea de regresión verdadera

Deficiencias de la regresión lineal

Predicción de valores

Regresión

Función base radial (FBR)

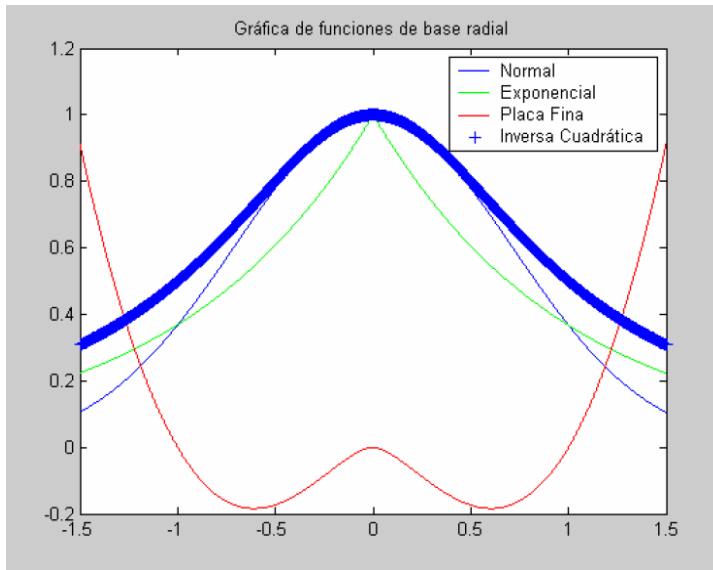
- Suma ponderada de un conjunto de funciones no lineales. Cada FBR se ajusta a regiones distintas del espacio de entrada
 - Se define una función $g_j(X)$ de aproximación a la función real $f(X)$

$$g_j(X) = \sum_{i=1}^m w_{ij} \phi_i(\|X - \mu_i\|)$$

- ϕ_i : función base radial
- w_{ij} : pesos
- vectores μ_i : centros
- Las funciones de base radial son simétricas respecto a su centro

Predicción de valores Regresión

Función base radial (FBR)



Funciones

- **Gaussiana**

$$\phi_i(z) = e^{-\varepsilon z^2}$$

- **Exponencial**

$$\phi_i(z) = e^{-z}$$

- **Cuadrática**

$$\phi_i(z) = z^2 + \alpha z + \beta$$

- **Cuadrática inversa**

$$\phi_i(z) = 1/\|1 + z^2\|$$

- **Placa fina**

$$\phi_i(z) = z^\alpha \log(z)$$

- **Trigonométrica**

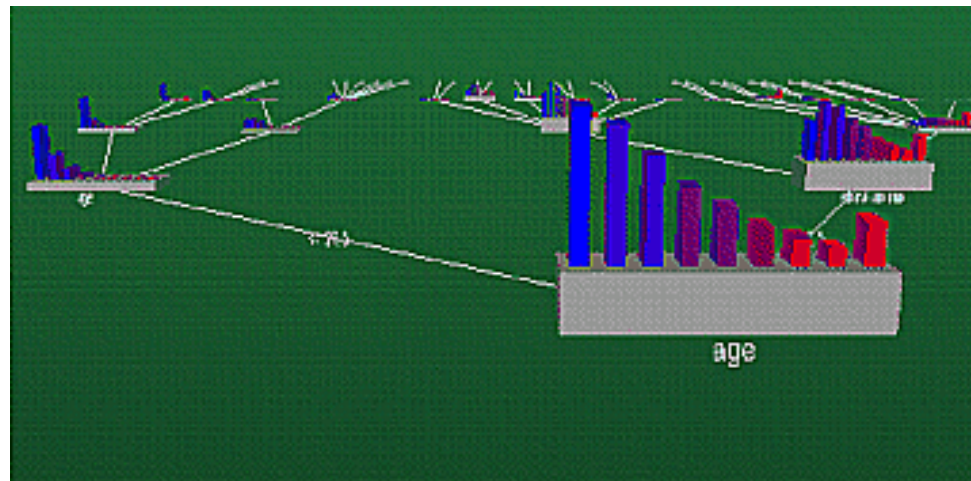
$$\phi_i(z) = \sin z$$

Predicción de valores

Regresión

Árboles de regresión

- Similares a los árboles de decisión, pero trabajan con etiquetas que tienen valores continuos
 - Los valores continuos del conjunto de entrenamiento se dividen en subrangos



Árbol de regresión

Predicción de valores

Regresión

Árboles de regresión

- **Criterio de partición:** Criterio para seleccionar el mejor atributo para dividir el conjunto de registros

- **Varianza:** se elige la partición que minimiza la varianza (V)

$$V = \frac{\sum_{i=1}^n (x_i - x_M)^2}{n}$$

- **Desviación absoluta:** se elige la partición que minimiza la desviación absoluta (D)

$$D = \frac{\sum_{i=1}^n (x_i - m)^2}{n}$$

x_M : media, m : mediana

- **Varianza normalizada:** es la varianza dividida por el logaritmo en base 2 del número de nodos hijos
- **Desviación absoluta normalizada:** es la desviación absoluta dividida por el logaritmo en base 2 del número de nodos hijos

Predicción de valores Series temporales

Una serie temporal es una secuencia de n observaciones de una o varias variables ordenadas cronológicamente



Predicción de valores

Series temporales

- Se representan como una secuencia de vectores numéricos recogidos a lo largo del tiempo
- La predicción de series de tiempo requiere la estimación de valores que van a aparecer en la secuencia, dados los valores anteriores y los valores actuales
- A medida que transcurre el tiempo, los nuevos valores obtenidos se van incorporando a la información disponible
- Un aspecto importante de las series temporales es que se descarta la información irrelevante, utilizando únicamente la necesaria para generar las predicciones deseadas

Predicción de valores

Series temporales

- Las series de tiempo se miden a unidades fijas de tiempo. Sus valores pueden representarse como:
 $t(1), t(2), \dots, t(i), \dots, t(n)$ donde $t(n)$ es el valor más reciente
- Los valores futuros están directamente relacionados con los valores anteriores. Dada una serie, se puede construir una *ventana o intervalo de tiempo* considerando p puntos de tiempo anteriores:

$t_0, t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9$

Intervalo = 5

- A partir de los intervalos especificados se transforman las series de tiempo en un formato de casos estándar

Predicción de valores Series temporales

- Los casos se especifican en función de un objetivo y una ventana de tamaño m
- Se puede calcular la **media móvil** (MM) para esos casos:

$$MM(i, m) = \frac{1}{m} \sum_{j=i-m+1}^i t(j)$$

Caso	F1	F2	F3	F4	F5	Objetivo
1	t0	t1	t2	t3	t4	t5
2	t1	t2	t3	t4	t5	t6
3	t2	t3	t4	t5	t6	t7
4	t3	t4	t5	t6	t7	t8
5	t4	t5	t6	t7	t8	t9

- Las características de una tendencia se pueden obtener comparando medidas recientes con otras anteriores:

$$t(i) - MM(i, m)$$

$$MM(i, m) - MM(i - k, m)$$

Bibliografía

- Breiman, L.: “Bagging predictors”. *Machine Learning*, 24(2), 123-140, 1996
- Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J. “Classification and Regression trees”, WadsWorth and Brooks/Cole, Monterey, 1984.
- Cestnik, G., Kononenko, I., Bratko, I. “Assistant 86: A knowledge acquisition Tool for Sophisticated users” in Bratko, I., Lavrac, N. (eds.) *Progress in Machine Learning*, Sigma Press, 1987.
- Freund, Y., Schapire, R.E. “Experiments with a new boosting algorithm”. *Proceedings 13th International Conference on Machine Learning*, 148-156, 1996.
- Gama, J., Brazdil, P. “Cascade Generalization”. *Machine Learning*, vol. 41, 3, 315-343, 2000.
- Ho, T.K. “The random subspace method for constructing decision forest”. *IEEE Transactions Pattern Analysis and Machine Intelligence*, vol. 20, 8, 832-844, 1998.
- Liu, B., Hsu, W., Ma, Y. Integration Classification and Association Rule Mining. *Proc. 4th Int. Conference on Knowledge Discovery and Data Mining*, pp. 80-86, 1998.
- Pearl, J. “Probabilistic Reasoning in Intelligent Systems”, *Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- Quinlan, J.R. “Induction of decision trees”. *Machine Learning*, 1(1):81–106, 1986.
- Quinlan, J.R. “C4.5: Programs for Machine Learning”. Morgan Kaufmann, San Mateo, CA., USA, 1993.
- Shalev-Shwartz, S. and Ben-David, S. “Understanding Machine Learning. From Theory to Algorithms”, Cambridge University Press, 2014.
- Wolpert, D.H. “Stacked Generalization”. *Neural Networks*, vol. 5, 241-259, 1992.